Dauphine | PSL
UNIVERSITÉ PARIS

M2 IASD Apprentissage
Université Paris Dauphine - PSL

Fast Matrix Factorization for Online Recommendation with Implicit Feedback

Implémentation MapReduce en PySpark

Yacine ABDELOUHAB
Paul JANIN

2026

Résumé

Un système de recommandation à feedback implicite n'observe que des clics ou des achats, jamais de notes. Plus de 99 % des couples utilisateur-item n'ont aucune interaction, et cette absence ne veut pas dire que l'item déplaît. L'algorithme eALS de He et al. tient compte de toutes ces entrées manquantes sans les parcourir, grâce à deux caches et à une mise à jour coordonnée par coordonnée. Il sait aussi prendre en compte une interaction nouvelle sans réapprendre le modèle entier.

Le papier n'en donne aucune implémentation MapReduce. Nous en écrivons deux, en PySpark et sur des RDD de paires seulement. Développées séparément, elles ne diffèrent que sur un point, ce qui reste distribué et ce qui remonte sur le driver. Nous ajoutons une initialisation par SVD randomisée, qui n'est pas dans le papier.

Sur le jeu de données Yelp, 25 677 utilisateurs et 696 865 interactions, eALS classe dans les cent premiers l'item que l'utilisateur a réellement choisi, pour 20,7 % des utilisateurs. Recommander simplement les items les plus populaires n'y arrive que pour 6,1 %. Le coût d'une itération reste tenable jusqu'à $K = 1024$, quand la référence MLlib ALS décroche dès 512. Et l'initialisation par SVD fait gagner quinze itérations sur vingt.

Table des matières

1 Introduction **4**

2 Formalisation du problème de recommandation **5**
2.1 Données . . . 5
2.2 Modèle à vecteurs latents . . . 6
2.3 Fonction de coût . . . 6

3 Les trois résultats de eALS **8**
3.1 Le poids des entrées manquantes . . . 8
3.2 Les caches . . . 9
3.3 La mise à jour coordonnée par coordonnée . . . 9
3.4 Les deux algorithmes . . . 11
3.5 Un meilleur point de départ . . . 12

4 Implémentation distribuée avec les RDD **14**
4.1 Première implémentation . . . 14
4.1.1 Les RDD dont nous avons besoin . . . 14
4.1.2 Équation (2), les poids c_i . . . 15
4.1.3 Équation (3), les caches . . . 15
4.1.4 Équation (4), les vecteurs p_u . . . 16
4.1.5 Équation (5), les vecteurs q_i . . . 18
4.1.6 L'ordre des phases, piloté par le driver . . . 18
4.1.7 Critère d'arrêt . . . 19
4.1.8 Mise à jour online . . . 20
4.1.9 L'initialisation spectrale . . . 21
4.1.10 Combien de shuffles . . . 22
4.2 Seconde implémentation . . . 23
4.2.1 Construction des RDD groupés . . . 23
4.2.2 Les caches, par produit matriciel . . . 24
4.2.3 Mise à jour d'une phase, sans jointure . . . 24
4.2.4 Boucle d'apprentissage, collect et broadcast . . . 25
4.2.5 Analyse des shuffles . . . 26
4.2.6 Preuve d'exactitude . . . 26
4.2.7 Mise à jour online, par micro-batches . . . 26
4.3 Comparaison des deux implémentations . . . 27

5 Analyse expérimentale **29**
5.1 Dataset . . . 29
5.2 Environnement d'exécution . . . 30
5.2.1 Première implémentation . . . 30
5.2.2 Seconde implémentation . . . 30
5.3 Résultats, première implémentation . . . 30
5.3.1 Protocole . . . 30
5.3.2 Qualité selon le rang K . . . 31
5.3.3 Intérêt de l'initialisation par SVD randomisée . . . 32
5.3.4 Coût de la mise à jour online . . . 33
5.3.5 Bilan . . . 33
5.4 Résultats, seconde implémentation . . . 33
5.4.1 Protocole . . . 33
5.4.2 Passage à l'échelle en K . . . 34
5.4.3 Passage à l'échelle en nombre de cœurs . . . 34
5.4.4 Passage à l'échelle en taille des données . . . 35
5.4.5 Sensibilité aux hyperparamètres de pondération . . . 35
5.4.6 Effet de K sur la précision, une fois retuné . . . 36
5.4.7 Convergence de la précision . . . 37
5.4.8 Mise à jour online . . . 37
5.5 Ce que les deux campagnes établissent . . . 38

6 Exécution du projet **39**
6.1 Première implémentation . . . 39
6.2 Seconde implémentation . . . 40
6.2.1 WSL . . . 40
6.2.2 Docker . . . 40
6.2.3 Les notebooks d'annexe . . . 41

7 Conclusion **42**
7.1 Ce que l'implémentation a demandé de comprendre . . . 42
7.2 Ce que les expériences montrent . . . 42
7.3 Prolongements . . . 43

Chapitre 1

Introduction

Un système de recommandation cherche à proposer à chaque utilisateur des contenus ou des objets pertinents, par exemple un film, un produit, une musique ou un article. Dans la suite, on appelle ces contenus ou ces objets des **items**.

Dans de nombreuses applications, les utilisateurs ne donnent aucune note explicite. Le système observe uniquement des actions telles qu'un clic, une vue ou un achat, c'est le **feedback implicite**. Une interaction observée représente alors un signal positif. En revanche, l'absence d'interaction ne signifie pas que l'utilisateur n'aime pas l'item, mais seulement qu'aucune action n'a été enregistrée.

Le modèle doit donc prendre en compte les paires utilisateur-item non observées sans les considérer comme des négatifs certains. Or ces paires représentent plus de 99 % de la matrice. L'algorithme eALS de He et al. [1] résout ce problème en intégrant toutes les entrées manquantes dans l'apprentissage sans les parcourir une par une. Le papier propose également une variante *online*, capable d'intégrer une nouvelle interaction sans réentraîner entièrement le modèle.

Le papier montre que l'algorithme peut être parallélisé, mais ne fournit aucune implémentation MapReduce. Notre travail consiste donc à développer une version PySpark et à conduire une analyse expérimentale sur le dataset Yelp utilisé par les auteurs.

Ce travail a été mené en parallèle par deux implémentations distinctes de eALS en PySpark, développées indépendamment et exécutées dans des environnements différents. Plutôt que de n'en retenir qu'une, ce rapport présente les deux : les implémentations diffèrent dans leur conception, et le code n'a pas tourné sur les mêmes environnements, ce qui en fait une comparaison intéressante à documenter. Le chapitre 4 détaille les deux conceptions RDD et les compare directement. Le chapitre 5 fait de même pour les résultats expérimentaux qu'elles produisent chacune.

Nous ajoutons enfin un élément qui ne figure pas dans le papier. L'algorithme part d'un tirage aléatoire, et rien ne l'impose. La section 3.5 construit un point de départ par SVD randomisée, la section 4.1.9 le distribue, et la section 5.3 mesure ce qu'il fait gagner.

Le rapport suit une progression des équations vers le code. Le chapitre 2 formalise le problème et fixe les notations. Le chapitre 3 énonce les trois résultats de eALS dont l'implémentation a besoin. Le chapitre 4 décrit les RDD retenus et le code qui les manipule. Le chapitre 5 rassemble les résultats expérimentaux, et le chapitre 6 indique comment lancer le projet.

Chapitre 2

Formalisation du problème de recommandation

2.1 Données

On dispose d'un ensemble d'utilisateurs, d'un ensemble d'items, et des interactions observées entre les deux.

Ensembles et matrice d'interaction

On note $\mathcal{U} = \{1, \dots, M\}$ l'ensemble des utilisateurs et $\mathcal{I} = \{1, \dots, N\}$ l'ensemble des items. La **matrice d'interaction** $R = \{r_{ui}\}_{\substack{1 \le u \le M \\ 1 \le i \le N}} \in \{0,1\}^{M \times N}$ est définie par

$$r_{ui} = \begin{cases} 1 & \text{si l'utilisateur } u \text{ a interagi avec l'item } i, \\ 0 & \text{sinon.} \end{cases}$$

Trois sous-ensembles reviennent en permanence dans la suite. Ils désignent les interactions effectivement observées.

Interactions observées

On note

$$\mathcal{R} = \{(u,i) \in \mathcal{U} \times \mathcal{I} \ : \ r_{ui} = 1\},$$

l'ensemble de toutes les **interactions observées**. Pour un utilisateur u fixé et un item i fixé, on note respectivement

$$\mathcal{R}_u = \{i \in \mathcal{I} : r_{ui} = 1\}, \qquad \mathcal{R}_i = \{u \in \mathcal{U} : r_{ui} = 1\}.$$

La quantité $|\mathcal{R}_u|$ est le nombre d'items avec lesquels u a interagi. Sur les données réelles, elle vaut quelques dizaines alors que N vaut plusieurs dizaines de milliers. C'est cet écart que toute l'implémentation cherche à exploiter.

Le jeu de données Yelp

Le jeu de données utilisé dans ce rapport donne les ordres de grandeur suivants.

$$M = 25\,677, \qquad N = 25\,815, \qquad |\mathcal{R}| = 696\,865.$$

La matrice R compte donc $M \times N = 6{,}6 \times 10^8$ entrées, dont seules 696 865 valent 1. Plus de 99 % des entrées sont nulles.

2.2 Modèle à vecteurs latents

Le modèle représente chaque utilisateur $u \in \mathcal{U}$ et chaque item $i \in \mathcal{I}$ par un **vecteur latent** dans un espace commun de faible dimension.

Vecteurs latents et score prédit

On fixe une dimension $K \in \mathbb{N}^*$, petite devant M et N. À chaque utilisateur u on associe un vecteur $p_u \in \mathbb{R}^K$, et à chaque item i un vecteur $q_i \in \mathbb{R}^K$. Le **score prédit** est leur produit scalaire,

$$\hat{r}_{ui} = p_u^\top q_i = \sum_{f=1}^{K} p_{uf}\, q_{if}.$$

On note $P \in \mathbb{R}^{M \times K}$ la matrice dont la ligne u vaut $p_u^\top$, et $Q \in \mathbb{R}^{N \times K}$ celle dont la ligne i vaut $q_i^\top$.

La matrice complète des scores vaut $\hat{R} = PQ^\top$. Elle n'est jamais construite en pratique, sa taille $M \times N$ étant prohibitive. Les scores sont calculés à la demande, uniquement sur les paires utiles.

2.3 Fonction de coût

Une interaction observée et une entrée manquante ne portent pas la même information. On les distingue par un poids.

Poids de confiance

À chaque paire (u, i) on associe un **poids de confiance** $\tilde{w}_{ui} > 0$,

$$\tilde{w}_{ui} = \begin{cases} w_{ui} & \text{si } (u,i) \in \mathcal{R}, \\ c_i & \text{sinon.} \end{cases}$$

Le poids w_{ui} mesure la confiance accordée à une interaction observée. Le poids c_i mesure celle accordée à une entrée manquante sur l'item i. Sa valeur est précisée en section 3.

Le point déterminant pour la suite est que c_i dépend de l'item seulement, jamais de l'utilisateur. C'est cette propriété qui rendra le calcul factorisable.

Fonction de coût

Les vecteurs latents sont appris en minimisant

$$\mathcal{L}(P, Q) = \sum_{u \in \mathcal{U}} \sum_{i \in \mathcal{I}} \tilde{w}_{ui} \left(r_{ui} - p_u^\top q_i \right)^2 + \lambda \left(\|P\|_F^2 + \|Q\|_F^2 \right), \tag{1}$$

où $\lambda > 0$ est le coefficient de **régularisation** et $\|\cdot\|_F$ la **norme de Frobenius**, soit $\|P\|_F^2 = \sum_u \|p_u\|_2^2$.

Cette fonction se lit en deux temps. Le premier terme mesure l'écart entre ce que le modèle prédit et ce que l'on observe, pour chaque paire utilisateur-item. Le second terme pénalise les vecteurs de grande norme, ce qui limite le surapprentissage et contrôle l'échelle des scores prédits.

Le rôle du poids $\tilde{w}_{ui}$ mérite d'être précisé. Sur une interaction observée, $r_{ui} = 1$, l'écart est fortement pondéré. Le modèle est donc poussé à prédire un score élevé. Sur une entrée manquante, $r_{ui} = 0$, l'écart est au contraire atténué par un poids plus faible.

Cette atténuation est essentielle. Sans elle, le modèle chercherait à prédire exactement zéro sur toutes les paires non observées, qui forment plus de 99 % de la matrice. Or une entrée manquante n'est pas un désintérêt avéré, c'est peut-être précisément l'item qu'il faudrait recommander. Un poids faible revient donc à dire que prédire un score élevé sur une paire non observée n'est que légèrement pénalisé. C'est ce qui laisse au modèle la liberté de proposer des items que l'utilisateur n'a pas encore vus.

Conserver les entrées manquantes dans la fonction de coût est donc indispensable. C'est aussi ce qui la rend coûteuse. La double somme de (1) porte sur les MN paires et non sur les seules interactions observées. Sur Yelp, cela représente $6{,}6 \times 10^8$ termes alors que $|\mathcal{R}| = 696\,865$ seulement correspondent à une interaction réelle. Plus de 99,8 % du calcul porte donc sur des entrées manquantes. C'est précisément ce coût que eALS supprime, et c'est l'objet de la section suivante.

Chapitre 3

Les trois résultats de eALS

L'implémentation n'a besoin que de trois résultats du papier. Ce chapitre les énonce et explique ce que chacun apporte. Les démonstrations figurent dans [1].

3.1 Le poids des entrées manquantes

Le premier résultat fixe la valeur de c_i introduite au chapitre 2.

Popularité et poids par popularité

La **popularité** de l'item i est sa fréquence dans les interactions observées,

$$f_i = \frac{|\mathcal{R}_i|}{|\mathcal{R}|}.$$

Le poids des entrées manquantes sur cet item est alors

$$c_i = c_0 \frac{f_i^{\alpha}}{\sum_{j \in \mathcal{I}} f_j^{\alpha}}, \tag{2}$$

où $c_0 > 0$ règle la masse totale accordée aux entrées manquantes et $\alpha \geq 0$ règle l'influence de la popularité.

Un item populaire a été vu par beaucoup d'utilisateurs. Si u n'a pas interagi avec lui, il l'a probablement écarté, et l'absence d'interaction devient un signal négatif informatif, auquel on donne un poids fort. Un item peu connu, à l'inverse, n'a été vu par presque personne. Si u n'a pas interagi avec lui, c'est sans doute qu'il ne l'a jamais rencontré, l'absence d'interaction n'apprend rien, et on lui donne un poids faible.

Le paramètre α règle l'écart entre ces deux situations. Pour $\alpha = 0$, tous les items reçoivent le même poids c_0/N, ce qui redonne le poids uniforme de Hu et al. [2]. eALS contient donc cette méthode comme cas particulier. Plus α augmente, plus le poids se concentre sur les items populaires. Le dénominateur de (2) sert quant à lui à normaliser. Il garantit $\sum_i c_i = c_0$ quelle que soit la valeur de α. Le paramètre c_0 fixe donc une masse totale, que α répartit ensuite entre les items.

Le dénominateur $|\mathcal{R}|$ de la popularité, lui, n'a aucun effet sur le résultat. En le sortant de la

puissance,

$$c_i = c_0 \frac{|\mathcal{R}|^{-\alpha} |\mathcal{R}_i|^{\alpha}}{|\mathcal{R}|^{-\alpha} \sum_j |\mathcal{R}_j|^{\alpha}} = c_0 \frac{|\mathcal{R}_i|^{\alpha}}{\sum_j |\mathcal{R}_j|^{\alpha}},$$

car $|\mathcal{R}|^{-\alpha}$ ne dépend pas de j et se met en facteur dans la somme. Utiliser les comptes bruts $|\mathcal{R}_i|$ plutôt que les fréquences donnerait donc exactement les mêmes c_i.

3.2 Les caches

Le deuxième résultat introduit deux matrices qui servent à précalculer certains termes. On les appelle des **caches**.

Caches S^q et S^p

On définit deux matrices carrées de taille $K \times K$,

$$S^q = \sum_{i \in \mathcal{I}} c_i \, q_i q_i^\top, \qquad S^p = \sum_{u \in \mathcal{U}} p_u p_u^\top. \tag{3}$$

On remarque que S^q ne dépend pas de l'utilisateur u, et que S^p ne dépend pas de l'item i. Chacune peut donc être calculée une seule fois, puis réutilisée pour tous les utilisateurs ou pour tous les items. Leur taille ne dépend d'ailleurs ni de M, ni de N, uniquement de K. Pour $K = 32$, une telle matrice occupe seulement 8 kilo-octets.

C'est ce qui allège le calcul. Ces deux matrices permettent de résoudre le problème d'optimisation sans parcourir, dans une somme, l'ensemble des N items ou des M utilisateurs. Seules les interactions observées restent à parcourir, et elles sont peu nombreuses. La contribution de toutes les entrées manquantes est lue directement dans le cache.

3.3 La mise à jour coordonnée par coordonnée

Le troisième résultat simplifie la résolution de (1), en deux temps.

Le premier temps rend le problème convexe. La fonction $\mathcal{L}(P, Q)$ n'est pas convexe en (P, Q) pris ensemble, on ne sait donc pas la minimiser directement. En revanche, si l'on fixe Q, elle devient convexe en P, et réciproquement. On alterne donc la résolution, en minimisant d'abord sur P à Q fixé, puis sur Q à P fixé.

$$P^* = \operatorname*{argmin}_{P \in \mathbb{R}^{M \times K}} \mathcal{L}(P, Q), \qquad Q^* = \operatorname*{argmin}_{Q \in \mathbb{R}^{N \times K}} \mathcal{L}(P, Q).$$

Chacun de ces deux problèmes se sépare ensuite entité par entité, car la loss est une somme de termes qui ne partagent aucune variable. Le problème sur P éclate en M problèmes indépendants, un par utilisateur, et celui sur Q en N problèmes indépendants, un par item.

$$\text{Pour chaque } u \in \mathcal{U}, \qquad p_u^* = \operatorname*{argmin}_{p_u \in \mathbb{R}^K} \sum_{i \in \mathcal{I}} \tilde{w}_{ui} \left(r_{ui} - p_u^\top q_i \right)^2 + \lambda \|p_u\|_2^2,$$

$$\text{Pour chaque } i \in \mathcal{I}, \qquad q_i^* = \operatorname*{argmin}_{q_i \in \mathbb{R}^K} \sum_{u \in \mathcal{U}} \tilde{w}_{ui} \left(r_{ui} - p_u^\top q_i \right)^2 + \lambda \|q_i\|_2^2.$$

C'est un problème classique, une régression des moindres carrés pondérée avec pénalité ridge. Sa solution s'écrit sous forme fermée. Toutefois, cette solution fermée demande d'inverser une matrice $K \times K$, pour chaque utilisateur et pour chaque item, ce qui est coûteux.

On peut supprimer cette inversion dans un second temps. Plutôt que de résoudre pour le vecteur p_u entier, on résout coordonnée par coordonnée, et de même pour q_i. On fixe toutes les coordonnées sauf une, et on minimise selon cette seule coordonnée. Pour cela on isole d'abord sa contribution au score prédit.

Prédiction partielle

Pour une coordonnée f fixée, on note

$$\hat{r}_{ui}^{\,f} = \sum_{k \neq f} p_{uk}\, q_{ik}$$

la prédiction obtenue sans la contribution de la coordonnée f.

Le problème devient alors scalaire.
Pour chaque $u \in \mathcal{U}$ et chaque $f \in \{1, \dots, K\}$,

$$p_{uf}^{*} = \operatorname*{argmin}_{p_{uf} \in \mathbb{R}} \sum_{i \in \mathcal{I}} \tilde{w}_{ui} \left(r_{ui} - \hat{r}_{ui}^{\,f} - p_{uf}\, q_{if} \right)^2 + \lambda\, p_{uf}^2,$$

Pour chaque $i \in \mathcal{I}$ et chaque $f \in \{1, \dots, K\}$,

$$q_{if}^{*} = \operatorname*{argmin}_{q_{if} \in \mathbb{R}} \sum_{u \in \mathcal{U}} \tilde{w}_{ui} \left(r_{ui} - \hat{r}_{ui}^{\,f} - q_{if}\, p_{uf} \right)^2 + \lambda\, q_{if}^2.$$

En développant le carré, chacune de ces expressions est un polynôme du second degré, de la forme $a\,x^2 + b\,x + c$. Le coefficient dominant a est strictement positif, le polynôme admet donc un minimum unique, atteint en $-b/2a$. Il n'y a plus de matrice à inverser, seulement une division.

Les K coordonnées sont mises à jour l'une après l'autre, et non simultanément. On calcule d'abord p_{u1}. Pour calculer p_{u2}, on utilise la valeur de p_{u1} qui vient d'être mise à jour, et ainsi de suite jusqu'à p_{uK}. Chaque coordonnée bénéficie donc immédiatement des précédentes. Une réécriture de ces deux expressions, n'utilisant dans les sommes que les éléments de $\mathcal{R}_u$ ou de $\mathcal{R}_i$ et faisant apparaître les caches de (3), donne enfin les formules suivantes.

Mise à jour d'une coordonnée

Toutes les autres coordonnées étant fixées, la valeur de p_{uf} minimisant (1) est

$$p_{uf} = \frac{\displaystyle\sum_{i \in \mathcal{R}_u} \left[w_{ui} r_{ui} - (w_{ui} - c_i) \hat{r}_{ui}^{\,f} \right] q_{if} \;-\; \sum_{k \neq f} p_{uk}\, S_{kf}^{q}}{\displaystyle\sum_{i \in \mathcal{R}_u} (w_{ui} - c_i) q_{if}^2 \;+\; S_{ff}^{q} + \lambda}. \tag{4}$$

Symétriquement, pour un item i et une coordonnée f,

$$q_{if} = \frac{\displaystyle\sum_{u \in \mathcal{R}_i} \left[w_{ui} r_{ui} - (w_{ui} - c_i) \hat{r}_{ui}^{\,f} \right] p_{uf} \;-\; c_i \sum_{k \neq f} q_{ik}\, S_{kf}^{p}}{\displaystyle\sum_{u \in \mathcal{R}_i} (w_{ui} - c_i) p_{uf}^2 \;+\; c_i\, S_{ff}^{p} + \lambda}. \tag{5}$$

Le numérateur oppose deux forces. Le terme $w_{ui} r_{ui}$ tire p_{uf} vers le haut quand les items connus de u s'expriment positivement sur la coordonnée f. Les termes en $\hat{r}_{ui}^{f}$ et S_{kf}^{q} tirent en sens inverse, ils pénalisent les prédictions non nulles là où aucune interaction n'est observée.

Le dénominateur est la courbure du polynôme minimisé. Ses deux sommes mesurent le poids que porte la coordonnée f, sur les items observés puis sur les manquants. Plus elle est contrainte par les données, plus il est grand, et plus la valeur retenue est prudente. Le terme λ garantit qu'il ne s'annule jamais.

Deux propriétés comptent ensuite pour la suite.

D'abord, les sommes ne portent que sur $\mathcal{R}_u$ ou $\mathcal{R}_i$, donc sur les seules interactions observées. Les entrées manquantes ont disparu, sans approximation.

Ensuite, les deux formules ne sont pas symétriques. Le facteur c_i apparaît devant le terme de cache dans (5), mais pas dans (4). La raison est que S^q est déjà pondéré par les c_i, alors que S^p ne l'est pas. Le détail compte, l'oublier fausse silencieusement la mise à jour des items.

3.4 Les deux algorithmes

En assemblant les trois résultats, on obtient l'apprentissage offline. Une itération met à jour tous les utilisateurs, puis tous les items. Les caches sont recalculés entre les deux phases, car P puis Q ont changé.

Algorithme 1, eALS offline

1. Initialiser aléatoirement P et Q, calculer les poids c_i par (2).
2. Calculer S^q et S^p par (3).
3. Répéter jusqu'à convergence.
   (a) Pour chaque utilisateur u, pour $f = 1, \ldots, K$, mettre à jour p_{uf} par (4).
   (b) Recalculer S^p.
   (c) Pour chaque item i, pour $f = 1, \ldots, K$, mettre à jour q_{if} par (5).
   (d) Recalculer S^q.

Le coût d'une itération est $O\left((M+N)K^2 + |\mathcal{R}|K\right)$.

La version online traite l'arrivée d'une interaction isolée. Réapprendre le modèle entier serait inutile, une interaction ne concernant directement que u et i.

Algorithme 2, mise à jour online des informations de la paire (u, i)

1. Poser $r_{ui} = 1$ et ajouter l'interaction à $\mathcal{R}$.
2. Sauvegarder $p_u^{\text{anc}} = p_u$, puis mettre à jour les K coordonnées de p_u par (4).
3. Corriger le cache, $S^p \leftarrow S^p - p_u^{\text{anc}} (p_u^{\text{anc}})^\top + p_u p_u^\top$.
4. Sauvegarder $q_i^{\text{anc}} = q_i$, puis mettre à jour les K coordonnées de q_i par (5).
5. Corriger le cache, $S^q \leftarrow S^q - c_i q_i^{\text{anc}} (q_i^{\text{anc}})^\top + c_i q_i q_i^\top$.

Les caches ne sont pas recalculés. Un seul vecteur ayant changé, il suffit de retirer son ancienne contribution et d'ajouter la nouvelle. Cette correction de rang un coûte $O(K^2)$. Le coût total d'une mise à jour online est $O\left((|\mathcal{R}_u| + |\mathcal{R}_i|)K + K^2\right)$, indépendant de M, de N et de $|\mathcal{R}|$.

3.5 Un meilleur point de départ

L'algorithme 1 commence par un tirage aléatoire, et le papier n'en dit pas davantage. Rien ne l'impose pourtant, et le coût d'une itération invite à chercher mieux. Cette section et la section 4.1.9 proposent donc un complément qui ne figure pas dans le papier étudié [1], un point de départ construit plutôt que tiré au hasard. Rappelons d'abord le problème que résout eALS,

$$(P^*, Q^*) \;=\; \operatorname*{argmin}_{P,\,Q} \sum_{u \in \mathcal{U}} \sum_{i \in \mathcal{I}} \tilde{w}_{ui} \left(r_{ui} - p_u^\top q_i \right)^2 \;+\; \lambda \left(\|P\|_F^2 + \|Q\|_F^2 \right),$$

où les $\tilde{w}_{ui}$ sont les poids définis plus haut. Considérons maintenant le problème obtenu en remplaçant tous ces poids par 1, et en retirant la régularisation,

$$(P^\circ, Q^\circ) \;=\; \operatorname*{argmin}_{P,\,Q} \|R - PQ^\top\|_F^2 \;=\; \operatorname*{argmin}_{P,\,Q} \sum_{u \in \mathcal{U}} \sum_{i \in \mathcal{I}} \left(r_{ui} - p_u^\top q_i \right)^2 .$$

Celui-là, contrairement au nôtre, admet une solution explicite.

Théorème de Eckart-Young

La solution du problème d'optimisation

$$\operatorname*{argmin}_{X \in \mathbb{R}^{M \times N},\ \operatorname{rang}(X) \le K} \|R - X\|_F$$

est la matrice

$$R_K \;=\; \sum_{j=1}^{K} \sigma_j \, u_j v_j^\top \;=\; U_K \, \Sigma_K \, V_K^\top,$$

où, pour tout $j \in \{1, \dots, K\}$, $\sigma_j \in \mathbb{R}_+$ est la j-ième valeur singulière de R, $u_j \in \mathbb{R}^M$ le j-ième vecteur singulier à gauche et $v_j \in \mathbb{R}^N$ le j-ième vecteur singulier à droite, avec

$$\sigma_1 \ge \sigma_2 \ge \cdots \ge \sigma_K > 0.$$

Les matrices U_K et V_K rassemblent ces vecteurs en colonnes, et $\Sigma_K = \operatorname{diag}(\sigma_1, \dots, \sigma_K)$.

Le minimum est donc atteint en R_K, et en posant

$$P^\circ = U_K \, \Sigma_K^{1/2}, \qquad Q^\circ = V_K \, \Sigma_K^{1/2},$$

on obtient $P^\circ \, (Q^\circ)^\top = R_K$.

Les deux problèmes ne diffèrent que par la pondération. Démarrer eALS en (P°, Q°) revient donc à le placer au minimum exact d'un problème presque identique au sien, et sans doute déjà dans la bonne région. Reste que le calcul explicite de R_K est hors de portée, puisqu'il demande R explicitement, soit 5,3 Go sur Yelp, pour un coût en $O(MN \min(M, N))$. C'est impensable.

Halko, Martinsson et Tropp [3] contournent l'obstacle. Leur algorithme n'accède jamais aux cases de R, uniquement à des produits de R par une matrice. L'idée tient en une observation. Multiplier R par un vecteur tiré au hasard donne un vecteur qui penche vers les directions de grande valeur singulière, puisque ce sont celles que R amplifie le plus. Quelques tirages suffisent alors à engendrer un petit sous-espace qui contient l'essentiel de R, et tout le calcul se ramène à ce sous-espace.

SVD randomisée de rang K, d'après Halko et al. [3]

1. Tirer Ω gaussienne de taille $N \times \ell$, avec $\ell = K + 10$, puis former $Y = R\,\Omega$. Chaque colonne de Y est une combinaison aléatoire des colonnes de R, elle penche donc vers les directions dominantes.
2. Orthonormaliser Y par le procédé de Gram-Schmidt, $Y = B\,T$. Les colonnes de B, de taille $M \times \ell$, forment alors une base orthonormée du même sous-espace.
3. Projeter la matrice R sur cette base, $C = B^\top R$, de taille $\ell \times N$. Elle ne compte plus que ℓ lignes au lieu de M.
4. Calculer la SVD exacte de C, ce qui est peu coûteux vu sa taille, et obtenir $C = \tilde{U}\Sigma V^\top$.
5. Comme $R \approx BB^\top R = B\,C = \left(B\tilde{U}\right)\Sigma V^\top$, poser $U = B\tilde{U}$, tronquer aux K premières colonnes, puis $P^\circ = U_K \Sigma_K^{1/2}$ et $Q^\circ = V_K \Sigma_K^{1/2}$.

L'intérêt pour nous est que R n'intervient qu'aux étapes 1 et 3, par des produits avec une matrice à ℓ colonnes. Ces deux produits se distribuent naturellement sous le paradigme MapReduce.

Chapitre 4

Implémentation distribuée avec les RDD

Le papier établit les équations (2) à (5) mais ne précise pas leur calcul distribué. Deux conceptions RDD en ont été développées, indépendamment l'une de l'autre. La première (section 4.1) reconstruit les regroupements par jointure à chaque itération. La seconde (section 4.2) distribue la structure des interactions une seule fois et ne rediffuse ensuite que les petits objets qui changent. La section 4.3 les compare directement.

4.1 Première implémentation

Nous en donnons une implémentation MapReduce en PySpark, en procédant équation par équation.

4.1.1 Les RDD dont nous avons besoin

Commençons par établir l'ensemble des RDD dont nous aurons besoin. De la matrice d'interaction R, on ne stocke que les entrées observées, un enregistrement par couple (u, i) tel que $r_{ui} = 1$. Les vecteurs p_u sont indexés par utilisateur et les q_i par item, chacun donne donc un RDD dont la clé est cette entité.

Nom	Élément	Signature exacte
`RDD_interactions`	(u, i, r_{ui}, w_{ui})	`RDD[(int,int,float,float)]`
`RDD_users`	(u, p_u)	`RDD[(int,ndarray[K])]`
`RDD_items`	$(i, (q_i, c_i, f_i))$	`RDD[(int,(ndarray[K],float,float))]`
S^q, S^p	matrices complètes sur le driver	`ndarray[K,K]`

TABLE 4.1 – État de l'implémentation

Les caches font exception. Pour $K = 32$, une matrice $K \times K$ pèse 8 Ko, une taille qui ne dépend ni de M, ni de N, ni de $|\mathcal{R}|$. Les distribuer n'aurait aucun sens. Ils restent donc entiers sur le **driver**.

Le driver construit le graphe d'exécution, garde les caches et décide de l'arrêt. Il ne calcule aucun vecteur. Les **exécuteurs** détiennent les partitions, reçoivent les caches par diffusion avant chaque phase, et font le calcul.

4.1.2 Équation (2), les poids c_i

La première équation à traduire fixe le poids des entrées manquantes.

$$c_i = c_0 \frac{f_i^\alpha}{\sum_j f_j^\alpha}, \qquad f_i = \frac{|\mathcal{R}_i|}{|\mathcal{R}|}, \quad |\mathcal{R}_i| = \operatorname{card}\{u \in \mathcal{U} : r_{ui} = 1\}.$$

Obtenir f_i demande de compter les interactions de chaque item, puis de diviser par leur nombre total. Le dénominateur de c_i, lui, porte sur tous les items et ne vaut qu'un seul nombre.

```
def item_et_un(interaction):
    u, i, r_ui, w_ui, *reste = interaction
    return (i, 1)

RDD_comptes = (RDD_interactions
    .map(item_et_un)
    .reduceByKey(lambda a, b: a + b, partitions))    # (i, |R_i|)

denominateur = (RDD_comptes.values()
    .map(lambda compte: (compte / nb_interactions) ** alpha)
    .sum())                                  # action, le driver recoit un
        float

RDD_items = RDD_comptes.map(build_item).partitionBy(partitions)
# build_item(i, |R_i|) -> (i, (q_i, c_i, f_i))
```

Code 4.1 – Initialisation des poids et des vecteurs items

Le comptage des $|\mathcal{R}_i|$ passe par `reduceByKey` plutôt que par `groupByKey`. L'addition étant associative, Spark l'applique déjà dans chaque partition, qui n'envoie alors qu'un compte partiel par item. `groupByKey` ferait traverser le réseau aux $|\mathcal{R}|$ valeurs 1 avant la moindre addition.

Le `sum` est une action. Il déclenche le plan et ramène un unique flottant sur le driver, ce qui est légitime puisque le dénominateur est commun à tous les items. Ce scalaire repart ensuite vers les exécuteurs dans la fermeture de `build_item`.

La valeur d'un item porte c_i et f_i à côté de q_i. Ces deux nombres ne changent plus après l'initialisation, et les garder ici évite une jointure à chaque itération.

Le `partitionBy` final redistribue les enregistrements de sorte qu'une clé tombe toujours dans la même partition, et Spark retient ce partitionneur. Il le compare avant chaque jointure et lit les partitions sur place quand les deux côtés portent le même. Un shuffle payé ici en évite un à chaque itération.

4.1.3 Équation (3), les caches

Les vecteurs q_i étant en place, on peut construire les deux caches.

$$S^q = \sum_i c_i q_i q_i^\top, \qquad S^p = \sum_u p_u p_u^\top.$$

Chaque ligne d'un RDD produit une matrice $K \times K$ et il ne reste plus qu'à les additionner. Le résultat pesant 8 Ko, les deux caches sont ramenés sur le driver.

```
def ajouter_item(Sq_partiel, ligne_item):
    i, (q_i, c_i, f_i) = ligne_item
    Sq_partiel += c_i * np.outer(q_i, q_i)
    return Sq_partiel

def ajouter_user(Sp_partiel, ligne_user):
    u, p_u = ligne_user
    Sp_partiel += np.outer(p_u, p_u)
    return Sp_partiel

def fusionner(gauche, droite):
    gauche += droite
    return gauche

Sq = RDD_items.treeAggregate(np.zeros((K, K)), ajouter_item,
   fusionner)
Sp = RDD_users.treeAggregate(np.zeros((K, K)), ajouter_user,
   fusionner)
```

Code 4.2 – Calcul des deux caches

Les deux appels ne diffèrent que par le poids c_i, présent dans S^q et absent de S^p. La fonction de fusion, elle, est commune.

`treeAggregate` est préférable à un `reduce`. Ce dernier ramènerait au driver une matrice partielle par partition, à charge pour lui de les additionner une à une, soit $P-1$ additions sur une seule machine. `treeAggregate` les combine deux à deux sur les exécuteurs et ne fait remonter que le sommet de l'arbre.

Le volume échangé ne dépend donc ni de M, ni de N, mais seulement de K et du nombre de partitions.

Une fois les caches sur le driver, il faudra les renvoyer aux exécuteurs pour la suite du calcul. C'est le rôle de `broadcast`, décrit à la sous-section suivante.

4.1.4 Équation (4), les vecteurs p_u

S^q redescend vers les exécuteurs par `broadcast`, qui n'en envoie qu'une copie par exécuteur au lieu d'une par tâche. Le cache change à chaque itération, la diffusion est donc refaite à chaque tour. Reste à réunir auprès de chaque utilisateur les termes de sa propre somme.

$$p_{uf} = \frac{\displaystyle\sum_{i \in \mathcal{R}_u} \left[w_{ui} r_{ui} - (w_{ui} - c_i) \hat{r}_{ui}^{\,f} \right] q_{if} \;-\; \sum_{k \neq f} p_{uk}\, S^q_{kf}}{\displaystyle\sum_{i \in \mathcal{R}_u} (w_{ui} - c_i) q_{if}^2 \;+\; S^q_{ff} + \lambda}.$$

La somme porte sur $\mathcal{R}_u$, donc sur les items de u. Elle réclame les quadruplets $(q_i, r_{ui}, w_{ui}, c_i)$, dont les deux premiers termes sont indexés par item et les deux autres par interaction. On prend donc l'item comme clé pour joindre les q_i, puis l'utilisateur pour regrouper.

```
def interaction_par_item(interaction):
    u, i, r_ui, w_ui, *reste = interaction
    return (i, (u, r_ui, w_ui))
```

```python
def observation_par_user(ligne):
    i, ((u, r_ui, w_ui), (q_i, c_i, f_i)) = ligne
    return (u, (q_i, r_ui, w_ui, c_i))

observations = (RDD_interactions
    .map(interaction_par_item)       # (i, (u, r_ui, w_ui))
    .join(RDD_items)                 # attache (q_i, c_i, f_i)
    .map(observation_par_user))      # (u, (q_i, r_ui, w_ui, c_i))

grouped = observations.combineByKey(
    creer, ajouter, fusionner, partitions)
    # (u, ([q_i], [r_ui], [w_ui], [c_i]))

RDD_users_nouveau = (grouped.join(RDD_users)
    .mapValues(mettre_a_jour_p_u)
    .partitionBy(partitions))
```

Code 4.3 – Plan RDD de la mise à jour des utilisateurs

Le `map` du milieu change la clé sans déplacer la moindre donnée. Chaque partition réécrit ses propres lignes, là où elles sont déjà. Le premier vrai déplacement vient du `combineByKey`, première opération à exiger que toutes les observations d'un même utilisateur soient réunies au même endroit.

Ce regroupement pourrait s'écrire avec `groupByKey`, mais `combineByKey` donne la main sur trois fonctions, la création de l'accumulateur, l'ajout d'une valeur dans une partition, et la fusion de deux accumulateurs partiels. Le résultat sort directement sous la forme de quatre listes alignées, prêtes pour NumPy, alors que `groupByKey` rendrait un itérateur brut qu'il faudrait reparcourir.

La dernière jointure, elle, ne coûte rien. `combineByKey` a réparti son résultat par la clé u, et `RDD_users` porte le même partitionneur depuis son `partitionBy`. Les deux RDD sont donc co-partitionnés, la ligne de u se trouve des deux côtés dans la même partition. Spark le détecte et lit les partitions correspondantes sans rien redistribuer.

Encore faut-il ne pas perdre cette propriété, et c'est le rôle du `mapValues` qui suit. Un `map` pourrait modifier les clés, Spark abandonnerait alors le partitionneur du résultat et la jointure de l'itération suivante redeviendrait un shuffle. `mapValues` garantit que les clés restent intactes et conserve le partitionneur.

La mise à jour d'un utilisateur

Le `mapValues` applique la fonction suivante **une fois par utilisateur**. Elle ne voit donc qu'une seule clé, avec les observations qui lui sont attachées, et ignore tout du reste.

L'emboîtement est le suivant. Un exécuteur détient plusieurs partitions, une partition contient de nombreux utilisateurs, et cette fonction est appelée pour chacun d'eux.

Elle ne contient plus aucun appel Spark et applique l'équation (4), une coordonnée après l'autre. Les tableaux `q_obs`, `r_ui`, `w_ui` et `c_i` sont les quatre listes sorties du `combineByKey`, avec une ligne par item de $\mathcal{R}_u$.

```python
prediction = q_obs @ p_u
for f in range(len(p_u)):
```

```
    q_f   = q_obs[:, f]
    sans  = prediction - p_u[f] * q_f
    num   = np.sum((w_ui*r_ui - (w_ui-c_i)*sans) * q_f)
    num  -= p_u @ Sq[:, f] - p_u[f] * Sq[f, f]
    den   = np.sum((w_ui-c_i) * q_f*q_f) + Sq[f, f] + lambda_reg
    ancien = p_u[f]
    p_u[f] = num / den
    prediction += (p_u[f] - ancien) * q_f
```

Code 4.4 – Balayage séquentiel des coordonnées de p_u

`prediction` contient les scores $\hat{r}_{ui}$ pour i dans $\mathcal{R}_u$. La variable `sans` en retire la part de la coordonnée courante, c'est le $\hat{r}^{f}_{ui}$ de la formule. Les variables `num` et `den` sont alors le numérateur et le dénominateur de (4), terme pour terme.

La ligne `num -= p_u @ Sq[:, f] - p_u[f] * Sq[f, f]` est la somme sur $k \neq f$. On calcule le produit complet, puis on retranche le terme $k = f$, ce qui évite une boucle Python sur les coordonnées.

La dernière ligne met à jour la prédiction au lieu de la recalculer. Sans elle, chaque coordonnée coûterait $O(|\mathcal{R}_u|K)$, soit $O(|\mathcal{R}_u|K^2)$ pour le vecteur entier. Elle rend en revanche la boucle séquentielle, la coordonnée suivante lisant ce qu'elle vient d'écrire. Le parallélisme est donc entre utilisateurs, pas entre coordonnées.

4.1.5 Équation (5), les vecteurs q_i

La phase items reprend le même plan avec les clés échangées.

$$q_{if} = \frac{\displaystyle\sum_{u \in \mathcal{R}_i} \left[w_{ui} r_{ui} - (w_{ui} - c_i)\hat{r}^{f}_{ui} \right] p_{uf} \;-\; c_i \sum_{k \neq f} q_{ik}\, S^{p}_{kf}}{\displaystyle\sum_{u \in \mathcal{R}_i} (w_{ui} - c_i) p_{uf}^2 \;+\; c_i\, S^{p}_{ff} + \lambda}.$$

On prend l'utilisateur comme clé pour joindre les p_u, puis l'item pour regrouper. La formule impose toutefois deux différences. Le cache lu est S^p, et le terme de cache est multiplié par c_i, parce que S^p n'est pas pondéré alors que S^q l'est.

L'accumulateur ne porte par ailleurs que trois listes au lieu de quatre. Le poids c_i est commun à toutes les interactions d'un même item, et il est déjà rangé dans `RDD_items`.

4.1.6 L'ordre des phases, piloté par le driver

Ces deux phases ne peuvent pas se recouvrir. La mise à jour des p_u lit S^q, qui dépend de Q. Si Q changeait pendant cette lecture, le cache deviendrait faux et la garantie de décroissance tomberait. Le driver les enchaîne donc, itération après itération.

```
for iteration in range(1, max_iterations + 1):

    # Phase utilisateurs, Q et S^q figes.
    RDD_users_precedent = RDD_users
    RDD_users = mettre_a_jour_users(
        RDD_interactions, RDD_users, RDD_items, Sq)
    RDD_users.persist(StorageLevel.MEMORY_AND_DISK)
    RDD_users.count()                # action, calcule et materialise
```

```
    RDD_users_precedent.unpersist()
    Sp = calculer_Sp(RDD_users, K)

    # Phase items, P et S^p figes.
    RDD_items_precedent = RDD_items
    RDD_items = mettre_a_jour_items(
        RDD_interactions, RDD_users, RDD_items, Sp)
    RDD_items.persist(StorageLevel.MEMORY_AND_DISK)
    RDD_items.count()
    RDD_items_precedent.unpersist()
    Sq = calculer_Sq(RDD_items, K)

    cout = objectif(RDD_interactions, RDD_users, RDD_items, Sq)
    if amelioration(cout) < seuil:
        break
```

Code 4.5 – La boucle d'apprentissage

Les fonctions appelées ici ne sont pas nouvelles. `mettre_a_jour_users` est le plan RDD de la sous-section précédente et `mettre_a_jour_items` son symétrique. `calculer_Sp` et `calculer_Sq` sont les deux agrégations de l'équation (3). Le test d'arrêt de la dernière ligne fait l'objet de la sous-section suivante.

Cette boucle vit sur le driver et ne contient aucun calcul. Chaque affectation construit un plan que Spark n'évalue pas, et les trois appels qui suivent demandent chacun une justification.

`persist` est nécessaire parce que `RDD_users` est lu deux fois, une fois pour bâtir S^p et une fois par la phase items. Un RDD n'étant pas mémorisé, Spark rejouerait sinon toute la lignée, jointures et shuffles compris, et le coût de la phase serait payé deux fois.

`count` rend cette persistance effective. `persist` n'est qu'une déclaration d'intention, le RDD n'étant réellement stocké qu'à sa première évaluation. L'action force cette évaluation avant les deux lectures, et fixe du même coup la position des partitions sur les exécuteurs qui viennent de les calculer.

`unpersist` libère la version précédente. Une itération ne modifie pas les vecteurs, elle en produit de nouveaux, et sans cette ligne l'empreinte mémoire croîtrait linéairement avec le nombre d'itérations.

4.1.7 Critère d'arrêt

À chaque itération, le coût $\mathcal{L}$ diminue. Le critère consiste à regarder de combien.

$$\delta_t = \frac{\mathcal{L}_{t-1} - \mathcal{L}_t}{\mathcal{L}_{t-1}}.$$

Si le coût passe de 1000 à 700, alors $\delta_t = 0{,}3$ et l'itération a retiré 30 % du coût. S'il passe de 1000 à 999, alors $\delta_t = 0{,}001$ et elle n'en a retiré que 0,1 %.

On divise par $\mathcal{L}_{t-1}$ pour ne pas dépendre de l'échelle. Une baisse de 100 est énorme quand le coût vaut 1000, et négligeable quand il vaut 10^6.

L'apprentissage s'arrête quand deux itérations de suite donnent un δ_t inférieur à 0,001. Le modèle ne progresse alors plus assez pour justifier un tour de calcul de plus. On en exige deux, car une seule peut venir du bruit numérique.

Encore faut-il pouvoir évaluer $\mathcal{L}_t$ sans construire les MN scores. La fonction de coût se réécrit pour cela sous forme creuse,

$$\mathcal{L}(P,Q) = \sum_{(u,i)\in\mathcal{R}} \left[w_{ui}(r_{ui} - p_u^\top q_i)^2 - c_i(p_u^\top q_i)^2 \right] + \sum_{u\in\mathcal{U}} p_u^\top S^q p_u + \lambda \left(\sum_u \|p_u\|_2^2 + \sum_i \|q_i\|_2^2 \right).$$

La première somme ne parcourt que `RDD_interactions`, et toute la contribution des $MN - |\mathcal{R}|$ entrées manquantes tient dans $p_u^\top S^q p_u$. Les trois sommes sont des agrégations distribuées, seul le scalaire $\mathcal{L}_t$ revient au driver.

4.1.8 Mise à jour online

Tout ce qui précède suppose un réapprentissage complet. À l'arrivée d'un micro-lot, on ne veut au contraire toucher qu'aux entités concernées.

$$S^p_{\text{nouv}} = S^p_{\text{anc}} - \sum_{u\in T_U} p_u^{\text{anc}} (p_u^{\text{anc}})^\top + \sum_{u\in T_U} p_u^{\text{nouv}} (p_u^{\text{nouv}})^\top. \tag{6}$$

```
# 1. Les utilisateurs touches par le lot, soit T_U.
T_U = (nouveaux_evenements
    .map(lambda evenement: (evenement[0], None))
    .reduceByKey(lambda a, _b: a, partitions))

# 2. La jointure sert de filtre, l'historique est restreint a T_U.
historique_touche = (historique
    .map(lambda inter: (inter[0], inter))
    .join(T_U)
    .values().map(lambda paire: paire[0]))

# 3. La MEME fonction que l'offline, sur ce sous-ensemble.
RDD_users_nouveau = mettre_a_jour_users(
    historique_touche, RDD_users, RDD_items, Sq)

# 4. Correction du cache, les deux sommes de l'equation ci-dessus.
anciens = RDD_users.join(T_U).mapValues(lambda paire: paire[0])
Sp = Sp - calculer_Sp(anciens, K) + calculer_Sp(RDD_users_nouveau, K)

# 5. Un RDD est immuable, on retire puis on ajoute.
RDD_users = (RDD_users.subtractByKey(T_U)
    .union(RDD_users_nouveau)
    .partitionBy(partitions))
```

Code 4.6 – Traitement d'un micro-lot

L'étape 3 est le point important. La fonction appelée est celle de la phase utilisateurs offline, sans la moindre modification. Il n'existe donc pas deux implémentations de eALS à maintenir.

La jointure de l'étape 2 sert de filtre. Un `filter` testant l'appartenance à une liste supposerait cette liste rapatriée sur le driver par un `collect`, puis rediffusée. La jointure garde tout distribué et se comporte de la même façon quelle que soit la taille du lot.

Le gain est celui annoncé par la formule. Corriger les caches coûte $O(|T_U|K^2)$ au lieu de $O(MK^2)$, un ordre de grandeur qui ne dépend plus du nombre total d'utilisateurs.

Le `partitionBy` de l'étape 5 est indispensable. Une union ne garantit pas le partitionneur de son résultat, et sans lui les jointures des lots suivants redeviendraient des shuffles. Une entité jamais vue reçoit par ailleurs son premier vecteur ici. C'est ce qu'on appelle le *cold start*, le cas d'un utilisateur ou d'un item sur lequel le modèle n'a encore rien appris.

4.1.9 L'initialisation spectrale

La section 3.5 propose de remplacer le tirage aléatoire par une SVD randomisée. Il reste à la distribuer.

Deux étapes seulement touchent R, l'esquisse $Y = R\,\Omega$ et la projection $C = B^\top R$. Écrites entrée par entrée, elles deviennent

$$(R\,X)_u = \sum_{i \in \mathcal{R}_u} X_i, \qquad (R^\top X)_i = \sum_{u \in \mathcal{R}_i} X_u.$$

Ce sont deux agrégations par clé, à ceci près que la clé est l'utilisateur dans un cas et l'item dans l'autre. Une seule fonction suffit donc pour les deux.

```
def produit_avec_R(RDD_interactions, X, taille, cle, autre):
    # cle = 0 donne R X, cle = 1 donne R^T X
    diffuse = sc.broadcast(X)

    lignes = (RDD_interactions
        .map(lambda inter: (inter[cle], diffuse.value[inter[autre]]))
        .reduceByKey(lambda a, b: a + b, partitions)
        .collect())

    resultat = np.zeros((taille, X.shape[1]))
    for identifiant, vecteur in lignes:
        resultat[identifiant] = vecteur
    return resultat
```

Code 4.7 – Les deux seuls produits qui touchent R

Le `map` émet, pour chaque interaction, la ligne de X portée par l'autre entité. Le `reduceByKey` additionne ces lignes par clé, en combinant déjà du côté map puisque l'addition est associative.

Le `collect` final ramène le résultat sur le driver. C'est légitime, il ne compte que $\ell = 26$ colonnes, soit 5,3 Mo pour Yelp.

X est diffusée par `broadcast`, une copie par exécuteur, que toutes ses tâches lisent en mémoire locale.

Le reste de l'algorithme, l'orthonormalisation de Gram-Schmidt puis la SVD de C, porte sur des matrices à 26 colonnes. Il s'exécute sur le driver, au même titre que les caches S^q et S^p.

Reste à situer le tout dans la boucle. L'initialisation spectrale remplace la première étape de l'algorithme 1, avant tout calcul de cache, et la suite est inchangée.

Algorithme 1, avec initialisation spectrale

1. Calculer les poids c_i par (2).
2. **Initialiser P et Q par SVD randomisée**, au lieu du tirage aléatoire.
3. Calculer S^q et S^p par (3).
4. Répéter jusqu'à convergence les deux phases habituelles.

Dans le code, un seul champ de configuration bascule d'un mode à l'autre.

```
"initialisation": "svd"          # ou bien "aleatoire"
```

Code 4.8 – Le choix se fait dans le fichier de configuration

4.1.10 Combien de shuffles

Un shuffle redistribue les enregistrements entre les partitions, par le réseau et par le disque. C'est l'opération coûteuse, comptons celles d'une itération.

Opération	Primitive	Shuffle
Phase utilisateurs		
Attacher les q_i	`join`	oui
Regrouper par u	`combineByKey`	oui
Attacher p_u	`join`	non
Recalculer S^p	`treeAggregate`	non
Phase items		
Attacher les p_u	`join`	oui
Regrouper par i	`combineByKey`	oui
Attacher q_i	`join`	non
Recalculer S^q	`treeAggregate`	non

TABLE 4.2 – Les huit opérations d'une itération offline

Une itération coûte donc **quatre shuffles**, deux par phase. Les quatre opérations gratuites le sont pour deux raisons déjà vues. Les secondes jointures trouvent leurs deux côtés co-partitionnés, le `combineByKey` qui précède ayant réparti son résultat sur la clé que le facteur joint porte déjà. Les deux agrégations, elles, n'échangent que des matrices $K \times K$ de 8 Ko.

Ce nombre ne dépend ni de M, ni de N, ni du numéro de l'itération. Seul le volume déplacé dépend de $|\mathcal{R}|$.

L'initialisation, elle, se paie une seule fois. Le comptage des $|\mathcal{R}_i|$ et les partitionnements en demandent quelques-uns dans les deux cas. L'initialisation spectrale y ajoute les deux produits avec R de la section 4.1.9.

	Aléatoire	SVD randomisée
Initialisation, une seule fois	0	2
Par itération	4	4

TABLE 4.3 – Shuffles selon l'initialisation retenue

Le total dépend donc du nombre d'itérations, que le chapitre 5 mesure.

Un micro-lot online déclenche les quatre mêmes shuffles, sur des RDD dont la taille est celle du lot. Traiter les événements par lots vaut donc mieux que déclencher une action Spark pour chacun, le coût de lancement des tâches étant fixe.

4.2 Seconde implémentation

Le choix directeur de cette seconde implémentation est de **distribuer la structure des interactions une seule fois**, et de ne plus jamais la retoucher ensuite. Seuls les petits objets qui changent à chaque itération, Q, P, S^q et S^p, sont redistribués, par broadcast plutôt que par jointure. Contrairement à la première conception, P **et** Q **ne sont pas des RDD** : ce sont deux tableaux NumPy, qui vivent sur le driver et sont broadcastés aux exécuteurs à chaque itération. Seule la structure d'appartenance, quel item appartient à quel utilisateur et réciproquement, est distribuée, et elle ne change jamais pendant l'apprentissage offline. Les caches et le vecteur de popularité c restent également sur le driver, comme dans la première conception.

4.2.1 Construction des RDD groupés

La première étape transforme la liste brute des paires (u, i) observées en deux RDD groupés, l'un indexé par utilisateur, l'autre par item, chacun associant à sa clé les items (ou utilisateurs) observés et leurs poids.

```
def build_views(pairs, n_parts):
    interactions = sc.parallelize(pairs).cache()

    def pack(items):
        items = sort(np.array(items))
        return items, np.ones_like(items), np.ones_like(items)  # (
            R_u, w, r)

    user_rdd = (interactions
        .groupByKey(n_parts)
        .mapValues(pack)
        .cache())

    item_rdd = (interactions
        .map(lambda pair: (pair[1], pair[0]))   # (i, u)
        .groupByKey(n_parts)
        .mapValues(pack)
        .cache())

    user_rdd.count(), item_rdd.count()
```

```
    interactions.unpersist()
    return user_rdd, item_rdd
```

Code 4.9 – Construction unique des vues utilisateur et item

Cette fonction n'est appelée **qu'une seule fois**, avant la première itération. Les poids valent uniformément 1 pour une interaction observée. Les deux `groupByKey` sont les deux seuls regroupements par clé de tout l'apprentissage offline, un coût payé une fois et amorti sur toutes les itérations qui suivent, là où la première conception rejoint `RDD_interactions` avec les q_i courants à chaque itération, puisque Q change à chaque tour. Distribuer un tableau diffusé ne coûte pas une jointure, il coûte un broadcast.

4.2.2 Les caches, par produit matriciel

Les deux caches se réécrivent comme des produits de matrices.

$$S^q = \sum_i c_i \, q_i q_i^\top = Q^\top \operatorname{diag}(c)\, Q, \qquad S^p = \sum_u p_u p_u^\top = P^\top P.$$

Un produit de cette forme, à savoir une matrice multipliée par sa transposée, s'appelle une **matrice de Gram** (pondérée pour S^q, ordinaire pour S^p). Le papier ne l'appelle pas ainsi, mais c'est le nom que porte la fonction `gram_matrix` ci-dessous, d'où la précision. La première écriture ajoute les N matrices $K \times K$ une par une, comme le `treeAggregate` de la première implémentation. La seconde obtient exactement la même somme par un unique produit matriciel, une opération que BLAS exécute bien plus vite qu'une boucle Python accumulant des produits extérieurs un par un.

```
def gram_matrix(Q_bcast, c_bcast, N, n_parts):
    bounds = chunk(N, n_parts)          # (n_chunks) intervals [start
        , end)

    def partial_block(bounds_chunk):
        start, end = bounds_chunk
        block = Q_bcast.value[start:end]
        weights = c_bcast.value[start:end]
        return (block * weights[:, None]).T @ block    # un seul
            produit matriciel

    return (sc.parallelize(bounds, n_parts)
        .map(partial_block)
        .treeReduce(lambda a, b: a + b))
```

Code 4.10 – Calcul de S^q par blocs, BLAS plutôt que boucle

Chaque tâche reçoit, par broadcast, un tableau Q déjà en mémoire et ne calcule qu'un produit matriciel sur son intervalle. `treeReduce` combine ensuite les résultats deux à deux, exactement comme le `treeAggregate` de la première implémentation. $S^p = P^\top P$ s'obtient par le même appel, sans le vecteur de poids.

4.2.3 Mise à jour d'une phase, sans jointure

Les deux RDD groupés étant en place, une phase se réduit à un `mapPartitions` sur un RDD déjà partitionné, qui ne lit que des tableaux broadcastés.

```
def user_phase(user_rdd, Q_bcast, Sq_bcast, c_bcast, lam):

    def on_partition(rows):
        Q, Sq, c = Q_bcast.value, Sq_bcast.value, c_bcast.value
        ids, vectors = [], []
        for u, (R_u, w_u, r_u) in rows:
            p_u = P_bcast.value[u].copy()
            q_obs = Q[R_u]                          # sans jointure
            prediction = q_obs @ p_u
            coordinate_sweep(p_u, prediction, q_obs, w_u, r_u, c[R_u
               ], Sq, lam)
            ids.append(u)
            vectors.append(p_u)
        return [(np.array(ids), np.array(vectors))]

    return user_rdd.mapPartitions(on_partition)
```

Code 4.11 – Une phase, en un seul mapPartitions

Aucun appel Spark n'apparaît à l'intérieur d'une partition : `Q[R_u]` est un accès tableau ordinaire dans la copie locale du tableau broadcasté, l'équivalent, dans cette conception, du `join` de la première. La phase items reprend exactement le même plan, avec $\mathcal{R}_i$, P et S^p à la place de $\mathcal{R}_u$, Q et S^q, et le facteur c_i devant le terme de cache.

4.2.4 Boucle d'apprentissage, collect et broadcast

Contrairement à la première implémentation, où P et Q restent des RDD d'une itération à l'autre, ce choix impose de ramener les vecteurs mis à jour sur le driver, de les intégrer au tableau complet, puis de broadcaster à nouveau ce tableau.

```
for iteration in range(1, max_iterations + 1):

    Sq = gram_matrix(Q_bcast, c_bcast, N, n_parts)
    Sq_bcast = sc.broadcast(Sq)

    results = user_phase(user_rdd, Q_bcast, Sq_bcast, c_bcast, lam).
       collect()
    P = merge(P, results)          # P[u] <- nouvelle valeur, pour
       chaque u
    P_bcast = sc.broadcast(P)

    Sp = gram_matrix(P_bcast, None, M, n_parts)
    Sp_bcast = sc.broadcast(Sp)

    results = item_phase(item_rdd, P_bcast, Sp_bcast, c_bcast, lam).
       collect()
    Q = merge(Q, results)
    Q_bcast = sc.broadcast(Q)
```

Code 4.12 – Une itération, sur le driver

`merge` recopie dans le tableau complet les seules lignes que la partition a effectivement mises à jour. Cette conception suppose que P et Q tiennent en mémoire sur le driver.

4.2.5 Analyse des shuffles

Une itération offline ne déclenche ici aucun shuffle de type jointure ou regroupement par clé. Les deux seuls `groupByKey` de tout l'apprentissage ont lieu à l'initialisation (section 4.2.1), une fois pour toutes les itérations qui suivent. Le calcul des caches est un `treeReduce`, la lecture de q_i un accès dans un tableau diffusé, la mise à jour un `mapPartitions` sur RDD déjà partitionné. Ce que chaque itération paie à la place, ce sont deux broadcasts de tableaux de taille $O((M+N)K)$ et deux collect() de même taille. Pour rappel, un `collect` n'est pas un shuffle entre exécuteurs au sens de Spark, mais il n'est pas gratuit pour autant, à comparer aux $O(|\mathcal{R}|)$ enregistrements que déplacerait une jointure.

4.2.6 Preuve d'exactitude

La section 4.2.1 affirme que distribuer les utilisateurs entre partitions ne change pas le résultat. Nous y dédions un test plutôt que de le postuler : sur un dataset synthétique, l'oracle séquentiel NumPy et la version Spark, avec la même seed, à 1, 3 puis 7 partitions, produisent des matrices P et Q identiques à 10^{-9} près. Effectivement, répartir les utilisateurs autrement ne change donc rien au résultat.

4.2.7 Mise à jour online, par micro-batches

L'algorithme 2 traite une interaction à la fois, et chaque mise à jour dépend de l'état laissé par la précédente. Il n'y a donc rien à paralléliser au sein même d'un micro-batch : le rôle de Spark se limite ici à l'ingestion du flux, pas au calcul. C'est une différence de nature avec la mise à jour par lot de la première conception (équation (6)), qui reste un calcul RDD distribué sur l'ensemble des entités touchées par le lot. Un utilisateur ou un item jamais vu à l'offline dispose déjà d'une ligne dans P ou Q, initialisée aléatoirement, ce qui règle le cold start.

```
def process_batch(batch_df, batch_id):
    for u, i, ts in batch_df.orderBy("ts").toPandas().itertuples():
        prior_count = state.n_prior_interactions(u)
        rank = rank_of_truth(state.scores_for(u), i)
        results.append((batch_id, u, i, prior_count, rank))
        state.apply_interaction(u, i)

query = (spark.readStream.format("json").schema(schema)
    .option("maxFilesPerTrigger", 1)      # un fichier = un micro-
       batch
    .load(stream_dir)
    .writeStream.foreachBatch(process_batch)
    .option("checkpointLocation", checkpoint_dir)
    .trigger(availableNow=True)
    .start())
query.awaitTermination()
```

Code 4.13 – Micro-batches pilotés par Spark Structured Streaming

Le flux est découpé en amont en un fichier par micro-batch. `foreachBatch` s'exécute sur le driver, la fermeture sur `state` y est donc sûre bien qu'il s'agisse nominalement d'une requête distribuée. Cela rend `foreachBatch` adapté ici, là où un puits distribué imposerait de sérialiser l'état à chaque événement.

Une remarque supplémentaire : le `orderBy("ts")` est utile ici, puisque Spark ne garantit pas l'ordre des lignes d'un micro-batch lu depuis des fichiers. Le protocole est ensuite celui du papier (§5.3), *teste puis entraîne* : le rang de l'item réel est calculé avec le modèle tel qu'il était avant l'événement, puis l'événement sert à la mise à jour.
La correction de cache appliquée ici est celle de l'algorithme 2 (chapitre 3), rang un, événement par événement plutôt que lot par lot comme le fait l'équation (6). Un test dédié compare le résultat à un re-calcul complet des caches et mesure un écart relatif de 10^{-15} : la correction est exacte.

Le coût d'un événement est $O\Big((|\mathcal{R}_u| + |\mathcal{R}_i|)K + K^2\Big)$, indépendant de M, de N et de $|\mathcal{R}|$, exactement la complexité annoncée par le papier pour l'algorithme 2 (§4.3).

4.3 Comparaison des deux implémentations

Les deux implémentations résolvent le même problème mais placent la frontière entre « distribué » et « driver » à un endroit différent, avec des conséquences directes sur le coût par itération.

	Section 4.1	Section 4.2
P, Q	RDD, un tuple par entité	tableaux NumPy, driver, broadcastés
Structure $\mathcal{R}_u$, $\mathcal{R}_i$	reconstruite par jointure chaque itération	regroupée une seule fois
Shuffles par itération	4	0
Coût à la place	—	diffusion + collecte, $O((M+N)K)$
Calcul des caches	`treeAggregate`, boucle de produits extérieurs	`treeReduce` sur un produit matriciel bloc par bloc (BLAS)
Hypothèse de taille	aucune, P et Q restent distribués	P et Q tiennent en mémoire driver
Mise à jour online	RDD distribué sur les entités touchées par le lot	état séquentiel driver, ingestion par Spark Structured Streaming
Validation	analytique	mesurée, 10^{-9} à 10^{-15}

Table 4.4 – Différences entre les deux conceptions

Le compromis tient est le suivant : la première implémentation garde P et Q distribués et paie une jointure à chaque itération, la seconde les ramène sur le driver et paie un broadcast plus un collect() à la place. Ce second choix suppose $(M+N)K$ borné, une hypothèse vraie sur Yelp jusqu'à $K = 1024$ mais qui cesserait de l'être sur un jeu dont M et N compteraient des

centaines de millions d'utilisateurs et d'items. Alors, le régime où garder P et Q distribués, comme le fait la première conception, redeviendrait nécessaire.

Chapitre 5

Analyse expérimentale

Les deux conceptions du chapitre 4 ont été mesurées séparément, chacune par exécution Spark sur le dataset Yelp complet, mais dans des environnements différents, la première (section 5.3) sous Docker et la seconde (section 5.4) sous WSL2. La section 5.5 porte des regards croisés sur ces deux implémentations.

5.1 Dataset

Les deux conceptions partent sur le dataset Yelp, exactement celui utilisé dans le papier, et qui n'est accessible que depuis le repo des auteurs. En effet, en visitant le site actuel de yelp, le dataset a été largement enrichi, ce qui n'aurait pas permis de reproduire les conditions expérimentales du papier.

Caractéristique	Valeur
Lignes brutes	731 671
Utilisateurs M	25 677
Items N	25 815
Interactions distinctes $\lvert\mathcal{R}\rvert$	696 865
Sparsité	99,89 %
Degré utilisateur médian / max / moyen	16 / 1 024 / 27,1
Degré item médian / max / moyen	12 / 1 309 / 27,0

Table 5.1 – Statistiques du dataset Yelp complet

Les deux implémentations mesurent sur ce dataset complet, la section 5.3 pour la première et la section 5.4 pour la seconde. Le degré d'un utilisateur est le nombre d'items qu'il a notés, soit $|\mathcal{R}_u|$, et le degré d'un item le nombre d'utilisateurs qui l'ont noté, soit $|\mathcal{R}_i|$.

La distribution des degrés est très déséquilibrée, la médiane valant 16 alors que le maximum dépasse 1 000. Ce déséquilibre a une conséquence directe dans le calcul distribué : après un regroupement par utilisateur, quelle qu'en soit la méthode, les partitions ne portent pas la même charge, quelques utilisateurs très actifs produisant des accumulateurs bien plus volumineux que la médiane.

Deux détails supplémentaires touchent la construction de R. D'abord, le filtre "au moins 10 interactions" annoncé par le papier ne s'applique en réalité qu'aux utilisateurs : tous en comptent

au moins 10, mais 9 655 items sur 25 815 (37 %) en comptent moins, jusqu'à 1. Ensuite, 30 023 lignes (4,1 %) sont des avis répétés, fusionnés en une seule interaction pour l'apprentissage offline mais conservés tels quels pour le flux online de la section 5.4, où un avis répété est un événement à part entière.

5.2 Environnement d'exécution

Les deux implémentations n'ont pas tourné dans le même environnement.

5.2.1 Première implémentation

Toutes les mesures de la section 5.3 proviennent d'une image Docker, construite depuis le dépôt, et exécutée en local sur huit cœurs. L'image repose sur `python:3.11-slim-bookworm`, avec Java 17, PySpark 3.5.7 et NumPy 1.26.4. Aucune installation locale de Java ou de Spark n'est nécessaire.

5.2.2 Seconde implémentation

Toutes les mesures de la section 5.4 proviennent d'un environnement WSL2 sous Ubuntu 24.04 et versions suivantes des outils utilisés : Python 3.12, PySpark 4.2, Java 21. Toutes les expériences ont tourné sur une machine physique à 8 cœurs / 16 threads.

5.3 Résultats, première implémentation

5.3.1 Protocole

Le protocole est celui du papier, un *leave-one-out* temporel. Pour chaque utilisateur, la dernière interaction est cachée et sert de test, le reste sert à apprendre. Les utilisateurs de moins de trois interactions sont écartés. Il reste 671 187 interactions d'apprentissage sur 25 676 utilisateurs et 25 743 items.

Le modèle classe ensuite tout le catalogue pour un utilisateur u, privé des items que u a déjà notés. On note rang_u la position de l'item caché dans cette liste, et $\mathcal{T}$ l'ensemble des utilisateurs évalués. Deux mesures résument le résultat.

$$\text{HR@}k = \frac{1}{|\mathcal{T}|} \sum_{u \in \mathcal{T}} \mathbf{1}\left[\,\text{rang}_u \leq k\,\right], \qquad \text{NDCG@}k = \frac{1}{|\mathcal{T}|} \sum_{u \in \mathcal{T}} \frac{\mathbf{1}\left[\,\text{rang}_u \leq k\,\right]}{\log_2\left(\text{rang}_u + 1\right)}.$$

Le HR@100 compte les utilisateurs dont l'item caché entre dans les cent premiers. Le NDCG@100 compte les mêmes utilisateurs, mais pondère par le rang, si bien qu'un item classé troisième rapporte plus qu'un item classé quatre-vingt-dix-huitième. Le seuil de cent est celui du papier, et il ne signifie quelque chose que parce que le classement porte sur les 25 743 items du catalogue. Face à une poignée de candidats tirés au hasard, le HR@100 vaudrait presque 1 pour n'importe quel modèle.

Reste à fixer ce qui ne varie pas. Toutes les comparaisons qui suivent partagent ce découpage, cette règle de classement et les mêmes hyperparamètres, $c_0 = 512$, $\alpha = 0{,}4$, $\lambda = 0{,}01$ et $w_{ui} = 1$. Chaque entraînement reçoit vingt itérations, un budget et non un constat de convergence, le critère d'arrêt du chapitre 4 étant désactivé. Toutes les durées rapportées ici, enfin, sont mesurées sous Spark, en local sur huit cœurs.

5.3.2 Qualité selon le rang K

Les quatre modèles reçoivent le même budget de vingt itérations. Le point de comparaison est une recommandation par popularité, qui classe les items par f_i et propose donc la même liste à tout le monde.

Méthode	HR@100	NDCG@100	s / itération	Total (s)
Popularité	0,0608	0,0156	—	—
eALS, $K = 8$	0,1512	0,0377	30,4	623
eALS, $K = 16$	0,1833	0,0461	38,6	788
eALS, $K = 32$	**0,2068**	**0,0525**	49,6	1 021

TABLE 5.2 – Qualité des recommandations après vingt itérations

eALS donne à chaque utilisateur son propre vecteur p_u, donc son propre classement, et l'écart entre les deux mesure ce que rapporte cette personnalisation. Le meilleur modèle place l'item caché dans les cent premiers pour 20,7 % des utilisateurs, contre 6,1 % pour la popularité, soit un rapport de 3,4. Les deux mesures montent ensuite avec K sans exception, un espace latent plus large laissant davantage de place pour séparer les goûts.

Le prix se lit dans les deux dernières colonnes. Passer de $K = 8$ à $K = 32$ gagne 5,6 points de HR@100 et coûte 64 % de temps en plus, 1 021 secondes contre 623. Le coût par itération suit la croissance de K sans jamais la dépasser, ce qu'explique le $O\left((M + N)K^2 + |\mathcal{R}|K\right)$ du chapitre 3, le terme en $|\mathcal{R}|K$ dominant sur Yelp. Il reste dispersé, de 40 à 65 secondes à $K = 32$, le bruit d'ordonnancement de Spark en local pesant plusieurs secondes.

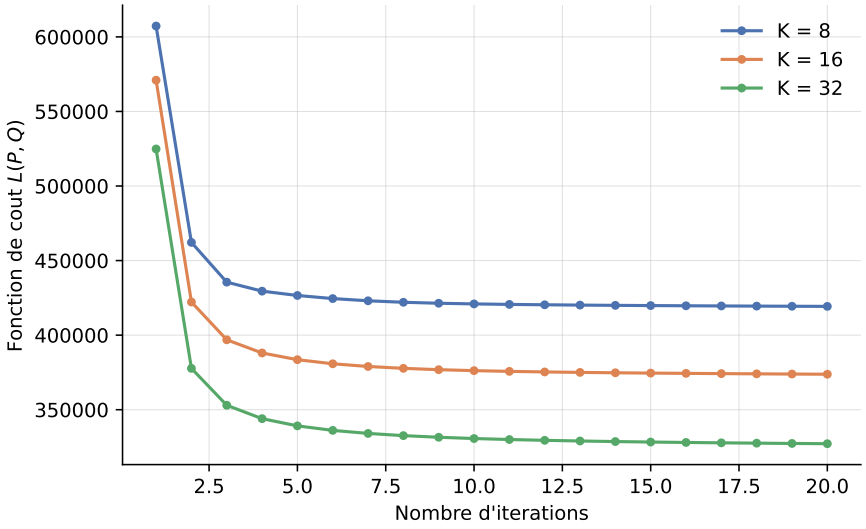


FIGURE 5.1 – Décroissance de la fonction de coût au fil des itérations

Les trois courbes s'aplatissent nettement avant la vingtième itération. Le budget est donc suffisant pour que la comparaison entre rangs porte sur des modèles comparables, et non sur des états d'avancement différents.

5.3.3 Intérêt de l'initialisation par SVD randomisée

Les deux entraînements comparés ici partagent tout, le rang $K = 16$, la graine, le découpage et les vingt itérations. Seul le point de départ change, un tirage aléatoire d'un côté, la SVD randomisée de la section 3.5 de l'autre.

Mesure	Aléatoire	SVD randomisée
Temps d'initialisation (s)	6,7	8,9
Temps total, initialisation comprise (s)	748	758
Fonction de coût après vingt itérations	373 788	**371 437**
HR@100	0,1833	0,1840
NDCG@100	0,0461	0,0463

TABLE 5.3 – Comparaison des deux initialisations, $K = 16$

La qualité finale ne bouge pas. Le HR@100 passe de 0,1833 à 0,1840 et le NDCG@100 de 0,0461 à 0,0463, des écarts qui ne signifient rien. Les deux entraînements convergent vers le même modèle, ce qui est attendu, une initialisation ne déplace pas l'optimum d'un problème.

Le gain est ailleurs, et il est net. La SVD part de 412 881 là où l'aléatoire part de 570 961, et son gain relatif à la première itération vaut 0,369 contre 0,149. Le modèle démarre déjà dans la bonne région au lieu de consacrer ses premières itérations à quitter l'origine.

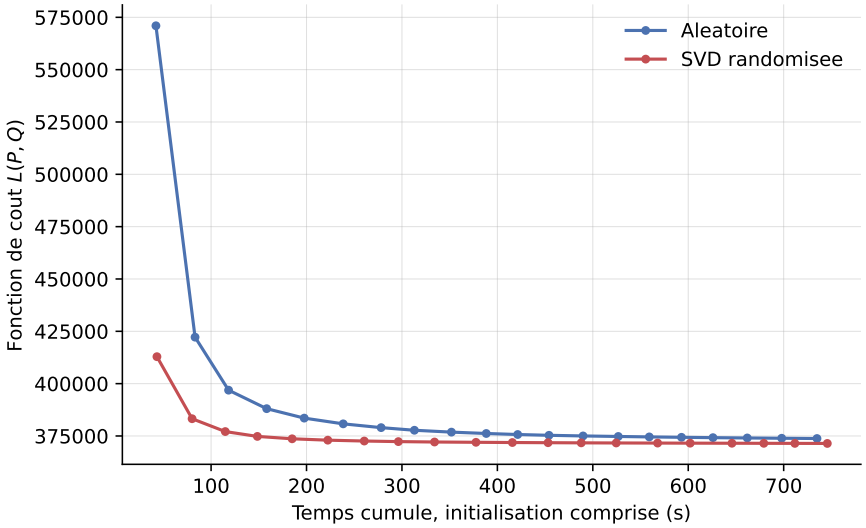


FIGURE 5.2 – Fonction de coût en fonction du temps cumulé, initialisation comprise

La figure met les deux coûts en face du temps réellement dépensé. Le niveau que l'aléatoire atteint au bout de ses vingt itérations, 373 788, est franchi par la SVD dès sa cinquième. Il a fallu 735 secondes à l'une et 185 à l'autre, soit un rapport de quatre.

Ce gain se paie très peu. L'initialisation aléatoire coûte 6,7 secondes, la SVD randomisée 8,9, l'écart de deux secondes couvrant les deux produits avec R et la décomposition sur le driver. Rapporté aux 550 secondes économisées, l'investissement est négligeable.

Le tableau 4.3 permet d'en tirer le décompte en shuffles. Avec l'initialisation aléatoire, qui n'en demande aucun, les vingt itérations en coûtent $20 \times 4 = 80$. Avec l'initialisation par SVD randomisée, qui en demande deux, les cinq itérations suffisantes en coûtent $2 + 5 \times 4 = 22$.

5.3.4 Coût de la mise à jour online

Le modèle appris à $K = 16$ reçoit ensuite cent nouvelles interactions, traitées en un seul lot par l'algorithme incrémental du chapitre 3.

Mesure	Valeur
Moteur	Spark, huit cœurs
Interactions ajoutées	100
Temps total de mise à jour (s)	29,06
Temps moyen par interaction (ms)	290,6

TABLE 5.4 – Traitement de cent nouvelles interactions

Ces 290 millisecondes ne mesurent pas le coût de l'algorithme. Par interaction, le chapitre 3 annonce quelques microsecondes,

$$O\left(\left(|\mathcal{R}_u| + |\mathcal{R}_i|\right) K + K^2\right).$$

Ce que le chronomètre voit est le coût fixe d'un lot Spark, ordonnancement et sérialisation compris, réparti sur cent événements seulement. Un lot plus gros l'amortirait d'autant.

La mesure établit malgré tout le point important. Ces 29 secondes ne dépendent ni de M, ni de N, ni du volume de $\mathcal{R}$. Seuls les utilisateurs et les items touchés par les nouveaux événements sont recalculés, ce qui est exactement la localité démontrée au chapitre 3.

5.3.5 Bilan

Trois enseignements se dégagent. Augmenter K améliore la qualité sans exception, au prix de 64 % de temps supplémentaire entre $K = 8$ et $K = 32$ pour 5,6 points de HR@100. L'initialisation spectrale ne change pas la qualité atteinte, mais divise par quatre le temps nécessaire pour l'atteindre, en échange de deux secondes. La mise à jour online traite cent interactions en 29 secondes, un chiffre dominé par le coût fixe d'un lot Spark et non par l'algorithme lui-même.

Ces mesures portent sur une seule machine, huit cœurs en local. Elles ne caractérisent ni le passage à l'échelle sur cluster, ni le coût réel des shuffles quand le réseau entre en jeu. Une campagne multi-exécuteurs reste la suite naturelle de ce travail.

5.4 Résultats, seconde implémentation

5.4.1 Protocole

Le protocole offline reprend le même *leave-one-out* que la section 5.3. Un second protocole, **online**, trie l'ensemble des interactions par date, entraîne sur les premiers 90 % et rejoue les derniers 10 % comme un flux (section 5.4.8). Sauf mention contraire, $\lambda = 0{,}01$ et $w_{ui} = 1$ pour les interactions observées. Toutes les mesures qui suivent proviennent d'exécutions Spark sur le dataset Yelp complet (tableau 5.1). La qualité y est rapportée en HR@10, et non en HR@100 comme à la section 5.3, ce dont la section 5.5 tire les conséquences.

5.4.2 Passage à l'échelle en K

Nous mesurons le temps par itération pour $K \in \{16, \ldots, 1024\}$, avec eALS et avec la référence Spark MLlib ALS (Hu et al.).

K	eALS (s/itér.)	MLlib ALS (s/itér.)	
16	1,08	0,48	
32	1,72	0,33	
64	3,00	0,61	
128	5,58	2,39	
256	11,45	14,68	
512	26,87	106,49	eALS 4× plus rapide
1024	74,88	—	

TABLE 5.5 – Temps par itération réel, Spark, dataset Yelp complet, `local[8]`

Le taux de croissance par doublement de K passe, pour eALS, de 2,0× entre 128 et 256 à 2,8× entre 512 et 1024. L'exposant monte de 1,0 vers 1,5, nettement sous-quadratique. Pour MLlib ALS, ce même taux vaut 6,1× puis 7,25×, un exposant proche de 3, la complexité cubique attendue d'une résolution par Cholesky d'un système $K \times K$ à chaque mise à jour. **En dessous de $K \approx 256$, MLlib l'emporte sur les facteurs constants**, son code Scala natif bat notre boucle Python malgré la complexité asymptotique moins favorable. Au-delà, l'écart s'inverse et se creuse vite d'ailleurs : à $K = 512$, eALS met 26,9 s quand MLlib en met 106,5, et n'a toujours pas terminé 5 itérations après 9 min 40 passées à $K = 512$ pour la tentative à $K = 1024$ (budget dépassé, mesure abandonnée). Cela fait écho à la Table 3 du papier, où l'ALS naïf de Hu et al. met 11,6 heures par itération à $K = 512$ sur Amazon.

5.4.3 Passage à l'échelle en nombre de cœurs

Nous répétons l'apprentissage à $K = 64$ sous `local[n]`, pour $n \in \{1, 2, 4, 8, 16\}$, chaque valeur de n redémarrant une session Spark propre.

Cœurs	eALS (s/itér.)	Accél.	MLlib (s/itér.)	Accél.
1	15,88	1,00×	3,25	1,00×
2	8,47	1,87×	1,67	1,95×
4	4,47	3,56×	0,92	3,53×
8	2,81	**5,65**×	0,59	**5,48**×
16	2,24	7,08×	0,68	4,80× (moins bon résultat donc)

TABLE 5.6 – Accélération réelle, dataset Yelp complet, seconde implémentation

L'accélération est quasi linéaire jusqu'à 8 cœurs, le nombre de cœurs physiques de la machine. À 16, qui sollicite l'hyper-threading sans cœur physique supplémentaire, eALS continue de progresser légèrement quand MLlib recule. C'est ici qu'on retrouve l'aspect "embarassingly parallel", mentionné dans le papier.

5.4.4 Passage à l'échelle en taille des données

À $K = 64$ fixé, nous sous-échantillonnons $|\mathcal{R}|$ à 10, 25, 50 et 100 %, sans réduire M ni N.

Fraction	$\lvert\mathcal{R}\rvert$ entraîné	Caches	Phase P	Phase Q	Total
10 %	55 621	0,23	1,18	0,89	2,33 s
25 %	150 894	0,16	1,27	1,14	2,58 s
50 %	322 891	0,15	1,22	1,25	2,62 s
100 %	671 188	0,17	1,29	1,27	2,75 s

TABLE 5.7 – Temps par itération quasi constant malgré $|\mathcal{R}|$ multiplié par 12, $K = 64$

Le temps par itération est quasi plat malgré un facteur 12 sur $|\mathcal{R}|$. Sous-échantillonner ne réduit ni M ni N, donc ne réduit presque pas le nombre d'utilisateurs "actifs" (avec au moins une interaction restante) que la boucle `mapPartitions` doit parcourir. À 27 interactions par utilisateur en moyenne, même 10 % des données laisse presque tous les utilisateurs avec au moins une interaction. Le coût, à ce K et cette sparsité, est donc dominé par le nombre d'appels Python (proportionnel à $M + N$) plutôt que par le volume de travail à l'intérieur de chaque appel (proportionnel à $|\mathcal{R}_u|$) : en pratique plus proche de $O\Big((M+N)K^2 + (M+N)K\Big)$ que du $O\Big((M+N)K^2 + |\mathcal{R}|K\Big)$ nominal, tant que $|\mathcal{R}|$ reste modeste devant $(M + N)$.

5.4.5 Sensibilité aux hyperparamètres de pondération

Nous reproduisons la méthodologie de la Figure 1 du papier : c_0 balayé à $\alpha = 0$, puis α balayé au meilleur c_0, à $K = 64$, 30 itérations.

c_0 ($\alpha = 0$)	HR@10	α ($c_0 = 2048$)	HR@10
8	0,0178	0,0	0,0410
32	0,0273	0,2	0,0421
128	0,0347	0,4	0,0421
512	0,0385	**0,5**	**0,0425**
2048	**0,0410**	0,6	0,0423
4096	0,0406	0,8	0,0365
8192	0,0387		

TABLE 5.8 – Recherche en grille, $K = 64$, optimum $c_0 = 2048$ et $\alpha = 0{,}5$

Une précaution méthodologique a compté ici : la grille initiale $\{8, 32, 128, 512, 2048\}$ était encore croissante à son bord droit, et aurait laissé croire que $c_0 = 2048$ est un optimum alors qu'il ne s'agissait que du dernier point testé. Étendre la grille à 4096 puis 8192 révèle le vrai sommet à $c_0 = 2048$.

Le sommet trouvé, $c_0 = 2048$, est environ 4× celui utilisé pour les mesures de la section 5.3 ($c_0 = 512$, la valeur rapportée par le papier pour Yelp). L'optimum en α, en revanche, $\alpha = 0{,}5$, est proche de celui du papier ($\alpha \approx 0{,}4$) et reproduit la même forme, en cloche, avec une chute nette après $\alpha = 0{,}6$.

Une vérification de convergence à 100 itérations (contre 30 pour la recherche en grille) donne HR@10 = 0,0420, essentiellement identique au point à 30 itérations : le modèle est donc déjà

convergé, ce qui écarte la sous-convergence comme explication des valeurs absolues obtenues ici, plus basses que celles du papier.

5.4.6 Effet de K sur la précision, une fois retuné

Le papier annonce (Figure 3) une précision qui croît avec K. Un premier test à $K = 128$, réutilisant l'optimum $(c_0, \alpha) = (2048,\ 0{,}5)$ trouvé à $K = 64$, donne HR@10 = 0,0345, moins bon que $K = 64$, mais ce test est injuste, il ne prouve que la non-transférabilité d'un optimum, pas la valeur de $K = 128$ lui-même.

Nous refaisons donc la recherche en grille de la section 5.4.5, intégralement, *à* $K = 128$.

Configuration	HR@10
$K = 64$, optimum propre ($c_0 = 2048$, $\alpha = 0{,}5$)	**0,0425**
$K = 128$, optimum de $K = 64$ réutilisé ($c_0 = 2048$, $\alpha = 0{,}5$)	0,0345
$K = 128$, optimum du papier ($c_0 = 512$, $\alpha = 0{,}4$)	0,0287
$K = 128$, optimum propre ($c_0 = 4096$, $\alpha = 0{,}2$)	0,0356

TABLE 5.9 – Comparaison à effort de recherche égal entre $K = 64$ et $K = 128$

Même avec son propre optimum ($c_0 = 4096$, deux fois celui de $K = 64$; $\alpha = 0{,}2$, une sensibilité à α nettement plus plate qu'à $K = 64$), $K = 128$ plafonne 16 % sous $K = 64$. Une vérification de convergence à 100 itérations confirme un plateau atteint dès l'itération 10, écartant là encore la sous-convergence.

Sur ce dataset et cette implémentation, davantage de facteurs n'améliore pas la précision, à effort de réglage égal entre les deux rangs. Ce constat contredit directement la Figure 3 du papier.

La section 5.3 observe pourtant l'inverse sur sa propre plage, la qualité y montant sans exception de $K = 8$ à $K = 32$. Les deux constats ne s'opposent pas nécessairement, ils portent sur des plages disjointes de K, mais rien ici ne permet de situer le rang où le retournement se produit.

5.4.7 Convergence de la précision

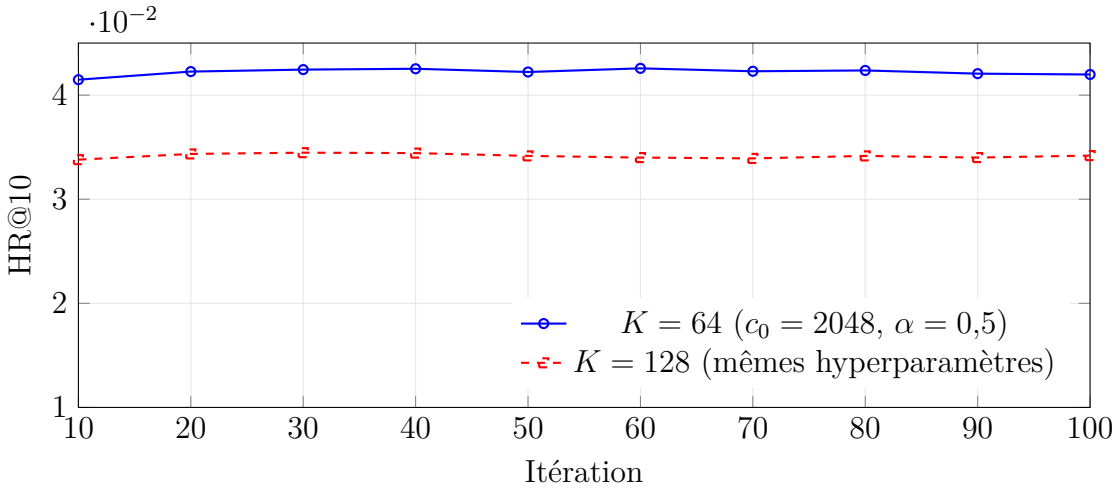


FIGURE 5.3 – HR@10 réel, dataset Yelp complet, mesuré tous les 10 itérations jusqu'à 100

Les deux courbes plafonnent dès l'itération 10 à 20 et restent stables ensuite, à ±0,0005 près, une variation compatible avec le bruit d'initialisation aléatoire plutôt qu'avec une tendance. C'est la preuve directe que les tableaux 5.8 et 5.9 comparent des modèles réellement convergés, pas des instantanés pris en cours d'apprentissage, à la différence de la figure 5.1, qui montre l'objectif encore en train de décroître au moment où l'apprentissage s'arrête.

5.4.8 Mise à jour online

Nous entraînons hors ligne sur les premiers 90 % des interactions (627 774, triées par date), puis rejouons les derniers 10 % (73 168 événements, non dédupliqués) via la pipeline Structured Streaming du chapitre 4.

Itérations online	HR@10	NDCG@10
0 (modèle figé)	**0,0348**	**0,0171**
1	0,0200	0,0075
2	0,0214	0,0080
3	0,0225	0,0085
5	0,0234	0,0088

TABLE 5.10 – 8 000 premiers événements du flux, $c_0 = 2048$, $\alpha = 0{,}5$, $w_{\text{new}} = 4$

Le résultat surprend : toute mise à jour online dégrade le HR@10 par rapport au modèle figé, l'inverse de la Figure 6 du papier. Nous avons cherché où le raisonnement pouvait casser avant de se fier à ce chiffre.

Le mécanisme fonctionne-t-il au moins sur un cas isolé ? Oui : pour un utilisateur et un item réels, entraîner sur cette interaction fait passer son rang de 395 à 110, immédiatement.
Reste l'hypothèse d'un poids w_{new} trop agressif, mais elle ne tient pas non plus : même à $w_{\text{new}} = 1$, où la nouvelle interaction compte exactement comme les anciennes, sans aucune sur-pondération, le HR@10 se dégrade encore, de 0,0348 à 0,0210.

L'explication tient dans une seconde métrique. En reprenant la même expérience à deux valeurs plus faibles de c_0 (l'hyperparamètre offline, jamais retuné pour l'usage online), le rang moyen donne un autre angle que le HR@10.

c_0 (offline)	Itér. online	HR@10	Rang moyen
512	0	0,0365	4 281,9
512	1	0,0241	**2 886,4**
128	0	0,0342	4 284,3
128	1	0,0281	**2 855,6**

TABLE 5.11 – Le rang moyen s'améliore malgré la dégradation du HR@10

Le rang moyen s'améliore d'environ 33 % **à chaque réglage testé**, alors même que le HR@10 se dégrade. La mise à jour online rapproche l'item cible du haut du classement *en moyenne*, sur 25 815 items, sans pour autant le faire entrer dans le top 10 de façon fiable, et peut même en déloger un autre qu'un modèle figé y laissait par hasard. L'algorithme 2 résout, à chaque événement, le problème aux moindres carrés *entier* sur les $K = 64$ coordonnées de p_u, contre tout $\mathcal{R}_u$. C'est un réajustement complet, plus sensible qu'un pas de descente à la pondération c_0 héritée de l'offline. L'écart en HR@10 se réduit d'ailleurs avec c_0 plus faible (-43 % relatif à $c_0 = 2048$, -34 % à $c_0 = 512$, -18 % à $c_0 = 128$), cohérent avec cette lecture.

Nous retenons ce résultat tel quel, c'est un constat vérifié à trois niveaux différents, sur un choix de conception (réutiliser un hyperparamètre offline sans le revalider online), plutôt qu'un défaut véritablement dû à l'algorithme lui-même.

5.5 Ce que les deux campagnes établissent

Les deux implémentations ont été mesurées sur le même dataset et sur la même classe de machine, huit cœurs. Leurs protocoles diffèrent pourtant, et il faut le dire avant de rapprocher quoi que ce soit.

Les temps par itération ne sont pas comparables en l'état. La section 5.3 chronomètre une itération complète, calcul de la fonction de coût inclus, ce qui ajoute à chaque tour une passe distribuée sur $\mathcal{R}$. La section 5.4 chronomètre les deux phases et les deux caches, sans ce calcul. À cela s'ajoute la différence de conception, quatre shuffles par itération d'un côté, aucun de l'autre. L'écart entre 38,6 et 1,08 seconde à $K = 16$ recouvre donc ces trois causes à la fois, et rien ici ne permet de les séparer.

Les mesures de qualité ne se recoupent pas davantage. La section 5.3 rapporte HR@100 et NDCG@100, le seuil retenu par le papier, la section 5.4 rapporte HR@10. Les deux séries se lisent séparément.

Ce que les deux campagnes établissent ensemble tient en deux points. La fonction de coût décroît à chaque itération dans les deux conceptions. Et la correction de cache par rang un est exacte, à $2{,}15 \times 10^{-16}$ pour la première et à 10^{-15} pour la seconde.

Chacune établit ensuite ce que l'autre ne mesure pas. La section 5.4 seule caractérise le passage à l'échelle, en K, en nombre de cœurs et en volume de données. La section 5.3 seule mesure l'apport de l'initialisation spectrale.

Chapitre 6

Exécution du projet

6.1 Première implémentation

Le code de cette implémentation se trouve dans le dossier `implementation1` du dépôt, et les commandes qui suivent se lancent depuis là. Tout tourne dans une image Docker construite sur place. Rien d'autre n'est à installer, ni Java, ni Spark, ni Python. Les mesures de la section 5.3 ont été obtenues dans l'environnement suivant.

Élément	Version
Système hôte	Windows 11, build 26 200
Docker	Docker Desktop 29,2,1, sur WSL2
Image de base	`python:3.11-slim-bookworm`
Java	`openjdk-17-jre-headless`
Spark	`pyspark 3.5.7`, en `local[*]`
NumPy	`1.26.4`
Cœurs alloués	8
Mémoire allouée	7,7 Gio

TABLE 6.1 – Environnement d'exécution

Six commandes rejouent l'ensemble. Les deux premières construisent l'image et lancent la suite de tests. Les trois suivantes exécutent une expérience chacune. La dernière trace les deux figures à partir des fichiers que ces expériences ont écrits.

```
docker build -t eals-1 .
docker run --rm eals-1 pytest -q

docker run --rm -v "$PWD/data:/data" eals-1 \
    python scripts/xp1_qualite.py --input /data/yelp.rating

docker run --rm -v "$PWD/data:/data" eals-1 \
    python scripts/xp2_initialisation.py --input /data/yelp.rating

docker run --rm -v "$PWD/data:/data" eals-1 \
    python scripts/xp3_online.py --input /data/yelp.rating

docker run --rm -v "$PWD/data:/data" eals-1 \
```

```
    python scripts/figures_job.py
```

Code 6.1 – Construction de l'image, tests, puis les trois expériences et les figures

Le seul argument à fournir est le fichier de données. Tous les réglages du rapport, les vingt itérations, la graine et les hyperparamètres, sont fixés dans le code. Chaque expérience affiche son tableau sur la sortie standard et écrit son détail dans un fichier JSON, que le script des figures relit ensuite.

Commande	Produit	Sortie du rapport	Durée
`docker build`	image	—	5 min
`pytest`	—	—	1 min
`xp1_qualite`	`xp1.json`	Tableau 5.2, figure 5.1	42 min
`xp2_initialisation`	`xp2.json`	Tableau 5.3, figure 5.2	25 min
`xp3_online`	`xp3.json`	Tableau 5.4	14 min
`figures_job`	`figures/`	Figures 5.1 et 5.2	5 s
Total			**≈ 1 h 30**

TABLE 6.2 – Ce que produit chaque commande, et sa durée

Les durées sont indicatives, mesurées sur la machine décrite plus haut. Elles sont dominées par les six entraînements de vingt itérations que les trois expériences enchaînent.

6.2 Seconde implémentation

6.2.1 WSL

```
curl -LsSf https://astral.sh/uv/install.sh | sh
uv venv --python 3.12 && uv pip install -e ".[dev]"

.venv/bin/pytest -q

.venv/bin/jupyter nbconvert --to notebook --execute --inplace \
    notebooks/00_correctness_check.ipynb notebooks/01_demo_small_data
       .ipynb
```

6.2.2 Docker

Une image Docker accompagne également cette seconde implémentation, pour la rendre reproductible sans dépendre de WSL2. Indépendante de WSL2, elle permet de rejouer la suite de tests et les notebooks sur toute machine dotée de Docker Engine, sans installation locale de Java ou de Spark.

```
docker build -t eals .
docker run --rm eals
```

```
docker run --rm eals jupyter nbconvert --to notebook --execute --
   stdout \
    notebooks/01_demo_small_data.ipynb
```

L'image installe Java 21, et les mêmes versions de PySpark, NumPy et pandas que la voie WSL2, pour que les deux environnements exécutent effectivement le même code.

6.2.3 Les notebooks d'annexe

Deux notebooks, déjà pré-exécutés accompagnent le code.

— `00_correctness_check.ipynb` reproduit, sur un problème synthétique de quelques secondes, la preuve d'exactitude de la section 4.2.6.

— `01_demo_small_data.ipynb` exécute la pipeline complète sur un petit échantillon, 300 utilisateurs réels tirés de Yelp, 4 955 items, 7 440 interactions, 176 Ko : chargement, apprentissage NumPy puis Spark, comparaison avec la référence MLlib ALS, mise à jour online isolée, puis via la pipeline Structured Streaming.

Chapitre 7

Conclusion

Le travail demandé était d'implémenter eALS en PySpark et de l'évaluer sur un jeu de données du papier. Deux conceptions indépendantes en sont issues (chapitre 4), mesurées dans des conditions différentes (chapitre 5), un fait qui structure aussi cette conclusion.

7.1 Ce que l'implémentation a demandé de comprendre

Traduire eALS en transformations RDD n'est pas une transcription mécanique. Les deux conceptions convergent sur certains points et divergent sur d'autres, ce qui est en soi instructif : cela signale lesquelles de ces décisions sont imposées par l'algorithme, et lesquelles relèvent d'un choix d'ingénierie.

Elles s'accordent d'abord sur le grain de parallélisme, auquel elles sont arrivées indépendamment. Les utilisateurs et les items sont indépendants, les coordonnées ne le sont pas. La boucle sur f reste séquentielle à l'intérieur d'un `mapValues` ou d'un `mapPartitions`, selon la conception, et confondre ces trois niveaux produirait un algorithme faux sans que Spark signale rien. La seconde implémentation l'a vérifié plutôt que postulé (section 4.2.6), l'oracle NumPy et la version Spark à 1, 3 et 7 partitions donnant des résultats identiques à 10^{-9} près. Elles s'accordent ensuite sur l'alternance, figer Q, mettre à jour tout P, recalculer le cache, puis seulement passer aux items. Ce séquencement n'est pas une contrainte technique de Spark, c'est ce qui garantit la décroissance de la fonction de coût.

Elles divergent sur un seul point, mais il commande tout le reste, où placer la frontière entre distribué et driver. La première garde P et Q comme des RDD et paie une jointure à chaque itération. La seconde les ramène sur le driver, les diffuse, et ne distribue qu'une seule fois la structure des interactions, qui ne change jamais. La section 4.3 détaille ce compromis, aucune des deux options n'étant strictement meilleure, chacune supposant une hypothèse de taille différente sur M, N et K. La même divergence se retrouve dans la mise à jour online, la première traitant un micro-lot comme un calcul RDD distribué sur les entités touchées, la seconde confiant à Spark la seule ingestion puisqu'un événement à la fois ne se parallélise pas. Les deux s'appuient pourtant sur la même correction de cache par rang un (équation (6)), à des granularités différentes.

7.2 Ce que les expériences montrent

Sur Yelp complet et sur Spark (section 5.4), eALS passe à l'échelle en K nettement mieux que la référence Spark MLlib ALS : sous-quadratique jusqu'à $K = 1024$ contre un exposant

proche de 3 pour MLlib, qui devient impraticable au-delà de $K = 512$. L'accélération réelle avec le nombre de cœurs atteint 5,6× sur 8 cœurs physiques, une mesure que le papier annonce possible sans jamais la fournir. Ces mesures viennent de la seconde implémentation, la seule des deux à avoir mené une campagne de passage à l'échelle (section 5.5).

Deux résultats vont à l'encontre des attentes du papier : d'abord, sur ce dataset, augmenter K de 64 à 128 dégrade la précision, même après avoir redonné à $K = 128$ sa propre recherche d'hyperparamètres à effort égal, l'inverse de la Figure 3 du papier. Ensuite, la mise à jour online dégrade le HR@10 par rapport à un modèle figé, l'inverse de sa Figure 6 ; l'examen du rang moyen révèle que la mise à jour améliore en réalité le classement en moyenne, et que c'est spécifiquement la précision au sommet de la liste qui en pâtit, plus sensible qu'attendu à un hyperparamètre de pondération réglé pour l'offline et jamais revalidé online. Les deux constats ont été vérifiés à plusieurs niveaux avant d'être retenus.

Un dernier résultat ne vient pas du papier. Remplacer le tirage aléatoire initial par une SVD randomisée, construite au chapitre 3 et distribuée au chapitre 4, ne change pas la qualité atteinte, 0,1840 de HR@100 contre 0,1833. Mais le niveau de coût que l'aléatoire atteint en vingt itérations est franchi dès la cinquième, soit 185 secondes au lieu de 735, pour deux secondes d'initialisation supplémentaires. Le gain est un gain de temps, pas de précision.

7.3 Prolongements

— Instrumenter l'exécution de la première conception pour remplacer son compte analytique de shuffles par une mesure réelle, et compléter le tableau 4.4.
— Mesurer l'accélération réelle sur un cluster multi-nœuds, au-delà du `local[n]` d'une seule machine physique.
— Retuner c_0 et α spécifiquement pour le régime online plutôt que de réutiliser l'optimum offline, pour vérifier si cela referme l'écart de la section 5.4.8.
— Traiter le déséquilibre de charge lié aux utilisateurs très actifs (degré maximal 1 024 contre une médiane de 16), par un partitionnement tenant compte du degré.
— Vectoriser la mise à jour sur tous les utilisateurs d'une partition à la fois, pour réduire le coût dominé par le nombre d'appels Python identifié section 5.4, plutôt que par le volume de travail par utilisateur.
— Étendre l'évaluation au second dataset du papier, afin de confronter directement nos chiffres aux siens.

Bibliographie

[1] X. He, H. Zhang, M.-Y. Kan, T.-S. Chua. *Fast Matrix Factorization for Online Recommendation with Implicit Feedback*. SIGIR 2016.

[2] Y. Hu, Y. Koren, C. Volinsky. *Collaborative Filtering for Implicit Feedback Datasets*. ICDM 2008.

[3] N. Halko, P.-G. Martinsson, J. A. Tropp. *Finding Structure with Randomness : Probabilistic Algorithms for Constructing Approximate Matrix Decompositions*. SIAM Review, 53(2), 217–288, 2011.